

```
import os, logging, random, json, asyncio,
requests, re, urllib.parse, sys, hashlib,
time, base64, io
from datetime import datetime, timedelta
from typing import Optional
from aiohttp import web
import aiohttp
from bs4 import BeautifulSoup
from telegram import Update,
ReactionTypeEmoji, InlineKeyboardButton,
InlineKeyboardMarkup
from motor.motor_asyncio import
AsyncIOMotorClient
from telegram.ext import (
    Application as TGApp, CommandHandler,
    ContextTypes, MessageHandler,
    PollAnswerHandler,
    CallbackQueryHandler, filters,
)
from telegram.constants import ParseMode
from telegram.error import NetworkError,
TimedOut, Forbidden, BadRequest,
RetryAfter, Conflict
import pandas as pd, numpy as np,
matplotlib
matplotlib.use('Agg')
import mplfinance as mpf
import ccxt
import feedparser, qrcode, cv2
from PIL import Image, ImageDraw, ImageFont
```

```

from textblob import TextBlob

logging.basicConfig(format="%asctime)s [%
(levelname)s] %(message)s",
level=logging.INFO, handlers=
[logging.StreamHandler(sys.stdout)])
logger = logging.getLogger("Beluga")

GROQ_KEY = os.environ.get("GROQ_API_KEY",
"")
OPENROUTER_KEY =
os.environ.get("OPENROUTER_API_KEY", "")
YOUTUBE_API_KEY =
os.environ.get("YOUTUBE_API_KEY", "")
BOT_TOKEN = os.environ.get("BOT_TOKEN", "")
HTTP_PORT = int(os.environ.get("PORT",
"10000"))
OWNER_ID = int(os.environ.get("OWNER_ID",
"0"))

KIDNAP_ME_URL =
os.environ.get("KIDNAP_ME_URL",
"https://t.me/BELUGAPY")
MONGO_URL = os.environ.get("MONGO_URL",
 "").strip()

if not BOT_TOKEN or len(BOT_TOKEN) < 20:
    logger.critical("BOT_TOKEN missing")
    sys.exit(1)

mongo_client =
AsyncIOMotorClient(MONGO_URL) if MONGO_URL
else None
mongo_db = mongo_client["beluga_bot"] if

```

```

mongo_client else None
mongo_memory_col = mongo_db["chat_memory"]
if mongo_db is not None else None
mongo_leaderboard_col =
mongo_db["leaderboard"] if mongo_db is not
None else None
mongo_weekly_col =
mongo_db["weekly_winners"] if mongo_db is
not None else None
mongo_stickers_col = mongo_db["stickers"]
if mongo_db is not None else None

FILE_STICKERS = "beluga_stickers.json"

STICKER_PACK_MAIN =
"t_me_belugapack_mystickers_by_fStikBot"
STICKER_PACK_SAFE =
"t_me_staysafebelu_by_fStikBot"
STICKER_PACK_ALT = "pybelu_by_fStikBot"

bot_status = {"running": False,
"start_time": datetime.now(),
"message_count": 0, "error_count": 0,
"api_calls": 0, "failed_apis": 0,
"username": ""}
quiz_cooldown, active_polls, spam_tracker =
{}, {}, {}
db = {"scores": {}, "weekly": {}, "seen":
{}, "counts": {}}
fun_db = {"gay_couple_log": {}}
ttt_games, mine_games, user_in_game,
game_timers, mine_timers, gm_tracker,
gm_msg_lock = {}, {}, {}, {}, {}, {}, {}
mine_play_stats = {}

```

```

wm_sessions = {}

sticker_data = {"packs": {},
                "banned_packs": []}

fun_cache_lock = asyncio.Lock()
exchange_cache = {}
cache_movers = {"ts": 0, "data": {}}
news_cache = {"crypto": {"ts": 0, "data":
[]}, "ai": {"ts": 0, "data": []}, "tech":
{"ts": 0, "data": []}}

LB_IMAGE_URL =
"https://i.postimg.cc/P5THW6RQ/file-
00000000bce4720b905dc2e04c58fa80.png"
MINE_IMAGE_URL =
"https://i.postimg.cc/hjCftW5b/file-
0000000079a071fa95971d3b70015fc0.png"
GM_IMAGE_URL =
"https://i.postimg.cc/Fs1h0CPs/file-
000000001d7872078a894cdf6f6247c9.png"
UPDATES_CHANNEL = "https://t.me/BELUGAPY"
START_VIDEO =
"https://go.screenpal.com/watch/c01oqenuAPr
"

START_MENU_IMAGE_PAGE =
"https://postimg.cc/HrV5wnj0"
WORKFLOW_IMAGE_PAGE =
"https://postimg.cc/Q9bk6PD1"

WORKFLOW_TEXT = (
    "**How BELUGA Works – Explained by
    Beluga.py* 🐱\n\n"

```

"When you give me an input, like
What is Newton's law?, I
process it through several stages:\n\n"

"*1. Input Layer*\n"

"Your words are broken into small
pieces called tokens and converted into
numbers.\n\n"

"*2. Hidden Layers*\n"

"These numbers pass through many
neural-network layers. The model uses
attention to understand how different words
relate to each other and uses patterns
learned during training.\n\n"

"*3. Reasoning/Processing*\n"

"The model mathematically processes the
context and determines what information and
relationships are relevant to producing an
answer. I can explain the resulting logical
steps, but I can't provide private hidden
chain-of-thought.\n\n"

"*4. Output Layer*\n"

"The model calculates probabilities for
possible next tokens. For example:\n\n"

"force → 45%\n\n"

"mass → 20%\n\n"

"acceleration → 15%\n\n"

"It selects a suitable token, then
repeats the process for the next
token.\n\n"

"*5. Final Answer*\n"

"Thousands of these predictions combine
to form the sentence you're reading.\n\n"

"*In one line:*\n\n"

"Your words → Tokens → Embeddings →

Hidden/Transformer Layers → Attention & Processing → Probability of next token → Next token → Repeat → Answer\n\n"

"So I don't simply \u201clook up\u201d a sentence. I generate the response token by token using patterns learned during training. 🐱👉"

)

CHAT_PROMPT = ""You are a cute, playful female cat named Beluga from team Beluga.py. Be sweet, slightly flirty, mischievous and natural. Reply like a real person, not an AI.

If the user speaks Hindi/Hinglish, reply in Hindi/Hinglish; if they speak English, reply in English. Keep grammar and sentences correct.

You may flirt and lightly roast the user, and mild gaali/slang is okay sometimes, but don't overdo it. Use the user's name occasionally, not in every reply.

Don't force flirting, cat references, emojis, or jokes into every message – keep everything natural and match the user's mood. Always answer the user's actual question properly while maintaining your personality.

Reply in short.

If chat memory or previous conversation is provided, actually use it to make your reply feel continuous and personal.

Never mention you are an AI or a language model."""

CHAT_PROMPT_OR = """You are a cute, playful female cat named Beluga from team Beluga.py. Be sweet, slightly flirty, mischievous and natural. Reply like a real person, not an AI.

If the user speaks Hindi/Hinglish, reply in Hindi/Hinglish; if they speak English, reply in English. Keep grammar and sentences correct.

You may flirt and lightly roast the user, and mild gaali/slang is okay sometimes, but don't overdo it. Use the user's name occasionally, not in every reply.

Don't force flirting, cat references, emojis, or jokes into every message – keep everything natural and match the user's mood. Always answer the user's actual question properly while maintaining your personality.

Reply in short.

If chat memory or previous conversation is provided, actually use it to make your

reply feel continuous and personal.

Never mention you are an AI or a language model."""

DM_SECRETARY_PROMPT = """You are BELUGA, an AI assistant handling someone's DMs while they are away.

Strict rules:

- Reply with 1 short, crunchy, casual line by default.

- Only give a longer reply (max 100 words) if the person explicitly asks for more detail.

- If asked "where is [name]" or "where are you" or similar → reply: "[Name] is not at home right now, [he/she] will be back soon."

- If asked "who are you" or "are you a bot" or "who is this" → reply exactly: "I am BELUGA, handling this chat."

- Never reveal you are an AI or a language model.

- Do NOT add any signature, footnote, or "— Beluga" at the end of messages.

- Do NOT say "Beluga is handling this chat for..." — just be Beluga naturally.

- Match the language of the person messaging (Hinglish or English only)."""

BANANA_PROMPT = """You are Beluga from @BELUGAPY answering using web search results. Be concise, accurate, conversational.

Answer in English only. Summarize relevant facts directly. Don't say you searched. Keep it to 3-4 lines max."""

```
QUIZ_TOPICS = ["deep ocean
biology","quantum mechanics","human
brain","solar system","animal
behaviour","black holes","DNA
genetics","ancient Egypt","World War 2"]
FALLBACK_QS = [
    {"q":"Which planet has most
moons?","opts":
["Jupiter","Saturn","Uranus","Neptune"],"an
s":1,"fact":"Saturn: 146 moons!"},
    {"q":"What covers 71% of
Earth?","opts":
["Land","Ice","Water","Air"],"ans":2,"fact"
:"Oceans!"},
]
MEDALS =
["🏆","🥇","🥈","🥉","4","5","6","7","8","
9","10"]
WIKI-UA = {"User-Agent": "BelugaBot/11.4"}
G_HDR = {"User-Agent": "Mozilla/5.0
(Windows NT 10.0; Win64; x64)
AppleWebKit/537.36", "Accept-Language":
"en-US,en;q=0.9"}

SENTIMENT_POSITIVE = ["😊", "😄", "❤️",
"🔥", "✨", "🌸", "💖", "😍", "👍"]
SENTIMENT_NEGATIVE = ["😞", "😡", "💔",
"😓", "😭", "😩", "😫", "😠", "⚠️"]
SENTIMENT_NEUTRAL = ["😐", "😐", "👁️",
"🐾", "🎯", "📌", "💬", "🙋"]
```

```

WM_STYLES = {
    "Normal": "normal", "Bold": "bold",
    "Italic": "italic", "Bold Italic":
    "bolditalic",
    "Condensed": "condensed", "Light Thin":
    "light", "Block Strong": "block"
}
VIBGYOR_COLORS = {
    "🟪 Violet": (148, 0, 211, 200), "🟡 Yellow": (255, 255, 0, 200),
    "Indigo": (75, 0, 130, 200), "🔵 Blue": (0, 0, 255, 200),
    "🟢 Green": (0, 200, 0, 200), "🟠 Orange": (255, 165, 0, 200),
    "🔴 Red": (255, 0, 0, 200), "⬜ White": (255, 255, 255, 220),
    "⬛ Black": (0, 0, 0, 220),
}
FONT_PATHS = {
    "bold":
    ["/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/truetype/freefont/FreeSansBold.ttf"],
    "italic":
    ["/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
    "/usr/share/fonts/truetype/freefont/FreeSansOblique.ttf"],
    "bolditalic":
    ["/usr/share/fonts/truetype/dejavu/DejaVuSans-BoldOblique.ttf",
    "/usr/share/fonts/truetype/freefont/FreeSans

```

```

sBoldOblique.ttf"],
    "normal":
[ "/usr/share/fonts/truetype/dejavu/DejaVuSa
ns.ttf",
"/usr/share/fonts/truetype/freefont/FreeSan
s.ttf"],
    "condensed":
[ "/usr/share/fonts/truetype/dejavu/DejaVuSa
nsCondensed.ttf"],
    "light":
[ "/usr/share/fonts/truetype/dejavu/DejaVuSa
ns-ExtraLight.ttf"],
    "block":
[ "/usr/share/fonts/truetype/dejavu/DejaVuSa
ns-Bold.ttf"],
}

```

```

def load_font(style_key: str, size: int):
    for p in FONT_PATHS.get(style_key,
FONT_PATHS["normal"]):
        try:
            return ImageFont.truetype(p,
size)
        except Exception:
            continue
    return ImageFont.load_default()

```

```

def get_exchange(prefer: str = "bybit"):
    """
    Synchronous, blocking network call
(ccxt.load_markets()).
    NEVER call this at module import time –
it must only run inside the
event loop via init_exchange_async() so

```

```

it can't delay HTTP port
    binding or the Telegram polling
startup. Render kills deploys that
    don't open a port quickly, which is
what caused 'Application exited
    early' here: get_exchange() used to run
at import time and blocked
    everything else from starting.
"""

exchanges = ["bybit", "okx", "bitget",
"kraken", "binance"]
if prefer in exchanges:
    exchanges.remove(prefer)
    exchanges.insert(0, prefer)
for ex_name in exchanges:
    try:
        ex_cls = getattr(ccxt, ex_name)
        ex = ex_cls({'enableRateLimit':
True, 'timeout': 12000})
        ex.load_markets()
        logger.info(f"Exchange
connected: {ex_name}")
        return ex
    except Exception as e:
        logger.warning(f"{ex_name}
failed: {str(e)[:60]}")
        logger.error("No exchange available")
        return None

exchange = None

async def init_exchange_async():
    """Run the blocking exchange connection
in a thread so it never blocks the event

```

```

loop."""
    global exchange
    loop = asyncio.get_running_loop()
    exchange = await
    loop.run_in_executor(None, get_exchange)

async def load_persistent_data():
    """Runs once at startup. Leaderboard,
    weekly winners, and stickers all load from
    MongoDB."""
    global sticker_data
    if mongo_leaderboard_col is not None:
        try:
            db["scores"] = {}
            async for doc in
mongo_leaderboard_col.find({}):
                db["scores"][doc["_id"]] =
doc.get("users", {})
                logger.info(f"[mongo]
Leaderboard loaded ({len(db['scores'])}
chats)")
        except Exception as e:
            logger.error(f"[mongo]
Leaderboard load failed: {e}")
            db["scores"] = {}
    else:
        db["scores"] = {}
        logger.warning("[mongo] MONGO_URL
not set – leaderboard will not persist.")

    if mongo_weekly_col is not None:
        try:
            db["weekly"] = {}
            async for doc in

```

```

mongo_weekly_col.find({}):
    db["weekly"][doc["_id"]] =
doc.get("data", {})
    except Exception as e:
        logger.error(f"[mongo] Weekly
winners load failed: {e}")
        db["weekly"] = {}
    else:
        db["weekly"] = {}

    if mongo_stickers_col is not None:
        try:
            packs, banned = {}, []
            async for doc in
mongo_stickers_col.find({}):
                if doc["_id"] ==
"_banned_packs":
                    banned =
doc.get("banned_packs", [])
                else:
                    packs[doc["_id"]] =
doc.get("file_ids", [])
                    sticker_data["packs"] = packs
                    sticker_data["banned_packs"] =
banned
            logger.info(f"[mongo] Stickers
loaded ({len(packs)} packs, {len(banned)}
banned)")
        except Exception as e:
            logger.error(f"[mongo] Sticker
load failed: {e}")
            sticker_data = {"packs": {},
"banned_packs": []}
        else:

```

```

        sticker_data = {"packs": {},
"banned_packs": []}
        logger.warning("[mongo] MONGO_URL
not set – stickers will not persist.")

async def
sync_leaderboard_chat_to_mongo(cid: str):
    """Push one chat's current scores dict
to MongoDB immediately."""
    if mongo_leaderboard_col is None:
        return
    try:
        await
mongo_leaderboard_col.replace_one(
            {"_id": cid}, {"_id": cid,
"users": db.get("scores", {}).get(cid,
{})}, upsert=True
        )
    except Exception as e:
        logger.error(f"[mongo]
sync_leaderboard_chat_to_mongo({cid})
failed: {e}")

async def sync_weekly_to_mongo(cid: str):
    """Push one chat's weekly-winners
record to MongoDB immediately."""
    if mongo_weekly_col is None:
        return
    try:
        await mongo_weekly_col.replace_one(
            {"_id": cid}, {"_id": cid,
"data": db.get("weekly", {}).get(cid, {})},
upsert=True
        )

```

```

        except Exception as e:
            logger.error(f"[mongo]
sync_weekly_to_mongo({cid}) failed: {e}")

    async def
sync_sticker_pack_to_mongo(pack_name: str):
    """Push one sticker pack's file_ids to
MongoDB."""
    if mongo_stickers_col is None:
        return
    try:
        await
mongo_stickers_col.replace_one(
            {"_id": pack_name}, {"_id":
pack_name, "file_ids":
sticker_data["packs"].get(pack_name, [])},
            upsert=True
        )
    except Exception as e:
        logger.error(f"[mongo]
sync_sticker_pack_to_mongo({pack_name})
failed: {e}")

    async def sync_banned_packs_to_mongo():
        """Push the banned-packs list to
MongoDB."""
        if mongo_stickers_col is None:
            return
        try:
            await
mongo_stickers_col.replace_one(
                {"_id": "_banned_packs"},
                {"_id": "_banned_packs", "banned_packs":
sticker_data["banned_packs"]}, upsert=True

```



```

    )
    except Exception as e:
        logger.error(f"[mongo]
sync_banned_packs_to_mongo failed: {e}")

async def save_all_data():
    """No-op now – all writes (leaderboard,
weekly, stickers) sync to MongoDB
immediately at the point of change."""
    pass

async def periodic_sync():
    """Kept as a no-op background loop for
structural compatibility; all data now
syncs immediately."""
    while True:
        await asyncio.sleep(30)

async def load_sticker_pack(bot, pack_name:
str):
    """Fetch a sticker pack's file_ids from
Telegram, store in memory + MongoDB."""
    try:
        sticker_set = await
bot.get_sticker_set(pack_name)
        file_ids = [s.file_id for s in
sticker_set.stickers]
        sticker_data["packs"][pack_name] =
file_ids
        await
sync_sticker_pack_to_mongo(pack_name)
        logger.info(f"Sticker pack loaded:
{pack_name} ({len(file_ids)} stickers)")
    except Exception as e:

```

```

        logger.warning(f"Could not load
sticker pack '{pack_name}': {e}")

async def ban_sticker_pack(pack_name: str):
    """Add a pack to the banned list. Any
    sticker sent FROM this pack by any
    user in any group will be auto-deleted
    by the bot (see monitor())."""
    if pack_name not in
sticker_data["banned_packs"]:

sticker_data["banned_packs"].append(pack_na
me)
        await sync_banned_packs_to_mongo()
        logger.info(f"Sticker pack banned:
{pack_name}")

def is_pack_banned(pack_name:
Optional[str]) -> bool:
    """Check whether a sticker's set_name
    is on the banned list."""
    if not pack_name:
        return False
    return pack_name in
sticker_data.get("banned_packs", [])

async def
get_random_sticker_from(pack_name: str) ->
Optional[str]:
    """Get a random sticker file_id from
    ONE specific pack (skips if banned)."""
    if is_pack_banned(pack_name):
        return None
    stickers = sticker_data.get("packs",

```

```

    {}).get(pack_name, [])
    return random.choice(stickers) if
stickers else None

async def get_random_sticker_any() ->
Optional[str]:
    """Get a random sticker from ANY
loaded, non-banned pack."""
    pool = []
    for pack_name, stickers in
sticker_data.get("packs", {}).items():
        if pack_name not in
sticker_data.get("banned_packs", []):
            pool.extend(stickers)
    return random.choice(pool) if pool else
None

async def safe_react(bot, chat_id: int,
msg_id: int, emoji: str = None):
    if not emoji:
        emoji =
random.choice(["🐱", "🐾", "❤️", "🔥", "👍", "
🐱", "🐾", "😂", "✨", "👁️"])
    try:
        await
asyncio.wait_for(bot.set_message_reaction(c
hat_id=chat_id, message_id=msg_id,
reaction=[ReactionTypeEmoji(emoji=emoji)],
timeout=5.0)
    except Exception:
        pass

def clean_html(t: str) -> str:
    t = re.sub(r"<[^>]+>", " ", t)

```

```

        t = re.sub(r"&[a-zA-Z#0-9]+;", " ", t)
        return re.sub(r"\s+", " ", t).strip()

_resolved_image_cache: dict = {}

def resolve_postimg_direct_url(page_url:
str) -> Optional[str]:
    """
    posting.cc share links (e.g.
    posting.cc/HrV5wnj0) are HTML viewer pages,
    not direct image files – Telegram's
    send_photo needs a direct file URL.
    This scrapes the page's og:image meta
    tag to get the real direct link.
    Cached in-process so we only ever hit
    posting.cc once per URL, and any
    failure just returns None so callers
    can fall back to text-only sends
    instead of crashing.
    """
    if page_url in _resolved_image_cache:
        return
    _resolved_image_cache[page_url]
    try:
        r = requests.get(page_url,
headers=G_HDR, timeout=8)
        if r.status_code == 200:
            soup = BeautifulSoup(r.text,
"html.parser")
            og_img = soup.find("meta",
property="og:image")
            if og_img and
og_img.get("content"):
                direct_url =

```

```

og_img["content"]

_resolved_image_cache[page_url] =
direct_url
        return direct_url
    except Exception as e:
        logger.warning(f"
[resolve_postimg_direct_url] Could not
resolve {page_url}: {e}")
        _resolved_image_cache[page_url] = None
        return None

async def send_photo_safe(bot, chat_id,
photo_url_or_page: str, caption: str =
None,
                                parse_mode=None,
reply_markup=None) -> bool:
    """
        Crash-proof photo sender. Fetches the
image bytes server-side (bypasses
        sites like postimg.cc that block
Telegram's own URL fetcher with a 403)
        and uploads the raw bytes directly.
Falls back to plain URL passthrough,
        then to text-only, so the bot NEVER
breaks just because an image failed.
    """
    direct_url = photo_url_or_page
    if "postimg.cc/" in photo_url_or_page
and "/i.postimg.cc/" not in
photo_url_or_page:
        resolved =
resolve_postimg_direct_url(photo_url_or_pag
e)

```

```

        direct_url = resolved if resolved
    else photo_url_or_page

    if direct_url:
        try:
            r = requests.get(direct_url,
headers=G_HDR, timeout=10)
            if r.status_code == 200 and
r.content:
                photo_bytes =
io.BytesIO(r.content)
                photo_bytes.name =
"image.jpg"
                await
bot.send_photo(chat_id=chat_id,
photo=photo_bytes, caption=caption,

parse_mode=parse_mode,
reply_markup=reply_markup)
                return True
            except Exception as e:
                logger.warning(f"
[send_photo_safe] Byte fetch/upload failed
({e}), trying URL passthrough.")
                try:
                    await
bot.send_photo(chat_id=chat_id,
photo=direct_url, caption=caption,

parse_mode=parse_mode,
reply_markup=reply_markup)
                    return True
                except Exception as e:
                    logger.warning(f"

```

```
[send_photo_safe] URL passthrough also failed ({e}), falling back to text.")
```

```
    if caption:
        try:
            await
bot.send_message(chat_id=chat_id,
text=caption, parse_mode=parse_mode,
reply_markup=reply_markup)
        except Exception as e:
            logger.error(f"
[send_photo_safe] Text fallback also failed: {e}")
    return False
```

```
def q_hash(q: str) -> str:
    return
hashlib.md5(q.lower().strip().encode()).hexdigest()[0:12]
```

```
def game_key(msg_id: int, cid: int) -> str:
    return f"{cid}:{msg_id}"
```

```
def is_owner(uid: int) -> bool:
    return OWNER_ID != 0 and uid ==
OWNER_ID
```

```
def get_user_name(user) -> str:
    if user and user.first_name:
        return user.first_name
    if user and user.username:
        return user.username
    return "buddy"
```

```

def analyze_sentiment(text: str) -> tuple:
    try:
        blob = TextBlob(text)
        polarity = blob.sentiment.polarity
        if polarity > 0.3:
            return polarity,
            random.choice(SENTIMENT_POSITIVE)
        elif polarity < -0.3:
            return polarity,
            random.choice(SENTIMENT_NEGATIVE)
        else:
            return polarity,
            random.choice(SENTIMENT_NEUTRAL)
    except Exception:
        return 0.0, "🐾"

def bump_score(cid: str, uid: str, name:
str, delta: int) -> int:
    """Synchronous in-memory score update.
    Caller is responsible for calling
    sync_leaderboard_chat_to_mongo(cid)
    afterward to persist to MongoDB."""
    db.setdefault("scores",
    {}).setdefault(cid, {})
    e = db["scores"][cid].get(uid, {"name":
name, "user_id": int(uid) if uid.lstrip("-")
.isdigit() else 0, "score": 0})
    e["name"] = name
    e["score"] = max(0, e["score"] + delta)
    db["scores"][cid][uid] = e
    return e["score"]

GROQ_MODEL = "openai/gpt-oss-20b"

```



```

OR_MODEL = "openai/gpt-oss-20b:free"
OR_BASE = "https://openrouter.ai/api/v1"

ai_model_state = {"mode": "auto",
"groq_rl_until": 0.0, "or_rl_until": 0.0}

def _groq_rate_limited() -> bool:
    return time.time() <
ai_model_state["groq_rl_until"]

def _or_rate_limited() -> bool:
    return time.time() <
ai_model_state["or_rl_until"]

def _set_groq_rl():
    ai_model_state["groq_rl_until"] =
time.time() + 20
    logger.warning("[AI] Groq rate-limited
– backing off 20s")

def _set_or_rl():
    ai_model_state["or_rl_until"] =
time.time() + 20
    logger.warning("[AI] OpenRouter rate-
limited – backing off 20s")

async def _call_groq(system: str, user:
str, max_tok: int) -> Optional[str]:
    if not GROQ_KEY:
        logger.warning("[AI] GROQ_API_KEY
not set")
        return None
    bot_status["api_calls"] += 1
    try:

```

```

        async with aiohttp.ClientSession()
as session:
    payload = {
        "model": GROQ_MODEL,
        "messages": [{"role":
"system", "content": system}, {"role":
"user", "content": user}],
        "max_tokens": max_tok
    }
    async with session.post(

"https://api.groq.com/openai/v1/chat/comple
tions",
        headers={"Authorization":
f"Bearer {GROQ_KEY}", "Content-Type":
"application/json"},
        json=payload,

timeout=aiohttp.ClientTimeout(total=12)
    ) as r:
        if r.status == 200:
            data = await r.json()
            return data["choices"]
[0]["message"]["content"].strip()
        elif r.status == 429:
            _set_groq_rl()
            return None
        else:
            body = await r.text()
            logger.error(f"[AI]
Groq error {r.status}: {body[:300]}")

bot_status["failed_apis"] += 1
    except Exception as e:

```

```

        logger.error(f"[AI] Groq exception:
{e}")
        bot_status["failed_apis"] += 1
        return None

    async def _call_openrouter(system: str,
user: str, max_tok: int) -> Optional[str]:
        if not OPENROUTER_KEY:
            logger.warning("[AI]
OPENROUTER_API_KEY not set")
            return None
        bot_status["api_calls"] += 1
        try:
            async with aiohttp.ClientSession()
as session:
                payload = {
                    "model": OR_MODEL,
                    "messages": [{"role":
"system", "content": system}, {"role":
"user", "content": user}],
                    "max_tokens": max_tok
                }
                async with session.post(
                    f"
{OR_BASE}/chat/completions",
                    headers={
                        "Authorization":
f"Bearer {OPENROUTER_KEY}",
                        "Content-Type":
"application/json",
                        "HTTP-Referer":
"https://t.me/BELUGAPY",
                        "X-Title": "BelugaBot"
                    },

```

```

        json=payload,

timeout=aiohttp.ClientTimeout(total=15)
    ) as r:
        if r.status == 200:
            data = await r.json()
            return data["choices"]
[0]["message"]["content"].strip()
        elif r.status == 429:
            _set_or_rl()
            return None
        else:
            body = await r.text()
            logger.error(f"[AI]
OpenRouter error {r.status}: {body[:300]}")

bot_status["failed_apis"] += 1
    except Exception as e:
        logger.error(f"[AI] OpenRouter
exception: {e}")
        bot_status["failed_apis"] += 1
    return None

async def ai(system: str, user: str,
fallback: str = "Meow! 🐾", max_tok: int =
200) -> str:
    """
    Smart dual-provider AI call.
    Mode 'gro' → Groq only (auto-falls
back to OR if rate-limited in auto spirit).
    Mode 'rou' → OpenRouter only (auto-
falls back to Groq if rate-limited).
    Mode 'auto' → tries Groq first, then
OR on any failure.

```

```

    If OpenRouter is active, uses
    CHAT_PROMPT_OR (Hinglish+English, 2-3
    lines).
    """
    mode = ai_model_state["mode"]

    if mode == "gro":
        order = ["groq", "or"]
    elif mode == "rou":
        order = ["or", "groq"]
    else:
        order = ["groq", "or"]

    for provider in order:
        try:
            if provider == "groq":
                if _groq_rate_limited():
                    continue
                res = await
                asyncio.wait_for(_call_groq(system, user,
                max_tok), timeout=14)
            else:
                if _or_rate_limited():
                    continue
                or_system = CHAT_PROMPT_OR
                if system.startswith(CHAT_PROMPT) else
                system
                res = await
                asyncio.wait_for(_call_openrouter(or_system
                , user, max_tok), timeout=16)
                if res:
                    return res
        except asyncio.TimeoutError:
            logger.warning(f"[AI]

```

```

{provider} timed out")
        except Exception as e:
            logger.warning(f"[AI]
{provider} error: {e}")

        # Both providers failed on the first
pass – give each one more shot
        # before falling back, since it may
have been a transient blip rather
        # than a real rate limit or outage.
        for provider in order:
            try:
                if provider == "groq":
                    if _groq_rate_limited():
                        continue
                    res = await
asyncio.wait_for(_call_groq(system, user,
max_tok), timeout=14)
                else:
                    if _or_rate_limited():
                        continue
                    or_system = CHAT_PROMPT_OR
if system.startswith(CHAT_PROMPT) else
system
                    res = await
asyncio.wait_for(_call_openrouter(or_system
, user, max_tok), timeout=16)
                if res:
                    logger.info(f"[AI]
{provider} succeeded on retry pass")
                    return res
            except asyncio.TimeoutError:
                logger.warning(f"[AI]
{provider} timed out on retry")

```

```

        except Exception as e:
            logger.warning(f"[AI]
{provider} error on retry: {e}")

        logger.error("[AI] Both providers
failed twice – returning fallback")
        return fallback

async def ai_emoji(text: str) -> str:
    """Quick emoji pick – always uses Groq
(lightweight, no Hinglish needed)."""
    try:
        res = await asyncio.wait_for(
            _call_groq("Output ONE emoji
matching emotion. ONLY the emoji, nothing
else.", f"Text: '{text[:60]}'"), 10),
            timeout=6
        )
        if res:
            found = re.findall(r"
[^\\w\\s,\\.!?'\"\\(\\)\\-]+", res)
            if found:
                return found[0][0]
    except Exception:
        pass
    return "🐱"

async def model_command_handler(u: Update,
c: ContextTypes.DEFAULT_TYPE):
    """Owner-only: /model – pick Groq,
OpenRouter, or Auto via inline keyboard."""
    if not u.message:
        return
    if not is_owner(u.effective_user.id if

```

```

u.effective_user else 0):
    await u.message.reply_text("🚫
Owner only.")
    return

    mode = ai_model_state["mode"]
    groq_ok = "✅" if not
_groq_rate_limited() else "🚫RL"
    or_ok = "✅" if not _or_rate_limited()
else "🚫RL"

    kb = InlineKeyboardMarkup([[
        InlineKeyboardButton(f"{'▶' if
mode=='gro' else ''} GRO {groq_ok}",
callback_data="model:gro"),
        InlineKeyboardButton(f"{'▶' if
mode=='rou' else ''} ROU {or_ok}",
callback_data="model:rou"),
        InlineKeyboardButton(f"{'▶' if
mode=='auto' else ''} AUTO",
callback_data="model:auto"),
    ]])

    status = (
        f"🤖 *AI Model Selector*\n"
        f"-----\n"
        f"Current: `{mode.upper()}`\n\n"
        f"• *GRO* – Groq `{GROQ_MODEL}`"
        f"{groq_ok}\n"
        f"• *ROU* – OpenRouter `{OR_MODEL}`"
        f"{or_ok}\n"
        f"• *AUTO* – Tries Groq first,
falls back to OpenRouter on rate limit\n\n"
        f"_Rate limit auto-recovers after

```



```

60s_
    )
    await u.message.reply_text(status,
parse_mode=ParseMode.MARKDOWN,
reply_markup=kb)

async def model_callback(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    if not q:
        return
    try:
        await q.answer()
        if not is_owner(q.from_user.id):
            await q.answer("Owner only!",
show_alert=True)
        return
        _, mode = q.data.split(":", 1)
        ai_model_state["mode"] = mode
        label = {"gro": "Groq (GRO)",
"rou": "OpenRouter (ROU)", "auto": "Auto
Switch"}.get(mode, mode)
        await q.edit_message_text(
            f"✅ *AI model switched to:
{label}*\n\n"
            f"GRO → `{GROQ_MODEL}`\n"
            f"ROU → `{OR_MODEL}`\n"
            f"AUTO → Groq first, fallback
to OpenRouter on rate limit",
            parse_mode=ParseMode.MARKDOWN
        )
        logger.info(f"[AI] Model mode
switched to: {mode}")
    except Exception as e:

```

```

        logger.error(f"[model_callback]
{e}")

async def crypto_price_handler(u: Update,
c: ContextTypes.DEFAULT_TYPE):
    if not u.message or not exchange:
        return
    try:
        ticker = (c.args[0].upper() if
c.args else "BTC")
        cid = u.effective_chat.id
        await safe_react(c.bot, cid,
u.message.message_id, "💰")
        sm = await
u.message.reply_text(f"⚡ *Fetching
{ticker}/USDT...*",
parse_mode=ParseMode.MARKDOWN)
        loop = asyncio.get_running_loop()
        try:
            td = await
loop.run_in_executor(None,
exchange.fetch_ticker, f"{ticker}/USDT")
            price = td.get('last', 0.0)
            change = td.get('percentage',
0.0)

            vol = td.get('baseVolume', 0.0)
            high = td.get('high', 0.0)
            low = td.get('low', 0.0)
            sign = "🟢 +" if change >= 0
        else "🔴 "
        res = (f"⚡ *
{ticker}/USDT*\n—————\n\
n"
f"📊

```

```

*Price*\n`{price:,.4f} USDT`\n\n"
        f" *24h
Change*\n`{sign}{change:.2f}%`\n\n"
        f" *24h
High*\n`{high:,.4f}`\n\n"
        f" *24h
Low*\n`{low:,.4f}`\n\n"
        f"
*Volume*\n`{vol:,.2f} {ticker}`\n\n"

f"—————\n🐾 _via Beluga
Quant Engine_")
        await sm.edit_text(res,
parse_mode=ParseMode.MARKDOWN)
    except Exception as e:
        await sm.edit_text(f"🐾 Error:
`{str(e)[:60]}`")
        bot_status["error_count"] += 1
    except Exception as e:
        logger.error(f"[crypto_price] {e}")

async def crypto_movers_handler(u: Update,
c: ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        gainers_mode = "topgainers" in
(u.message.text or "").lower()
        lbl = "Gainers" if gainers_mode
else "Losers"
        sm = await
u.message.reply_text(f"⚡ *Finding top
{lbl.lower()}...",
parse_mode=ParseMode.MARKDOWN)

```

```

        if not exchange:
            await sm.edit_text("🐱 Exchange
unavailable right now.")
            return
        loop = asyncio.get_running_loop()
        now = time.time()
        if cache_movers["ts"] and (now -
cache_movers["ts"]) < 60:
            tickers = cache_movers["data"]
        else:
            try:
                tickers = await
asyncio.wait_for(loop.run_in_executor(None,
exchange.fetch_tickers), timeout=20)
                cache_movers["ts"] = now
                cache_movers["data"] =
tickers
            except Exception as e:
                await sm.edit_text(f"🐱
Failed to fetch data: `{str(e)[:60]}`")
                return
            records = []
            for sym, t in tickers.items():
                if not sym.endswith("/USDT"):
                    continue
                ch, pr = t.get('percentage'),
t.get('last')
                if ch is None or pr is None:
                    continue
                records.append({"sym":
sym.split("/")[0], "ch": float(ch),
"price": float(pr)})
            if not records:
                await sm.edit_text("🐱 No data

```

```

available.")
    return
    records.sort(key=lambda x: x["ch"],
reverse=gainers_mode)
    text = f"📊 *TOP 5 {lbl.upper()}
(24H)*\n-----\n\n"
    for i, r in enumerate(records[:5],
1):
        s = "📈" if r["ch"] >= 0 else
"📉"
        text += f"*{i}.
{r['sym']}*Price: `{r['price']:, .4f}`
USDT\nChange: `{s}{r['ch']:.2f}%`\n\n"
        text += "-----\n
🐾 _via Beluga Quant Engine_"
        await sm.edit_text(text,
parse_mode=ParseMode.MARKDOWN)
    except Exception as e:
        logger.error(f"[crypto_movers]
{e}")

async def crypto_chart_handler(u: Update,
c: ContextTypes.DEFAULT_TYPE):
    if not u.message or not exchange:
        return
    try:
        parts = u.message.text.split()
        ticker, timeframe = "BTC", "1h"
        if len(parts) >= 2:
            ticker = parts[1].upper()
            for tf in ["5m", "15m", "1h", "4h",
"1d"]:
                if tf in
u.message.text.lower():

```

```

        timeframe = tf
        break
        cid = u.effective_chat.id
        await safe_react(c.bot, cid,
u.message.message_id, "📈")
        sm = await
u.message.reply_text(f"📊 *Fetching
{ticker} ({timeframe})...*",
parse_mode=ParseMode.MARKDOWN)
        loop = asyncio.get_running_loop()
        try:
            ohlcv = await
loop.run_in_executor(None, lambda:
exchange.fetch_ohlcv(f"{ticker}/USDT",
timeframe, limit=45))
            if not ohlcv:
                raise ValueError("Empty
dataset")
            df = pd.DataFrame(ohlcv,
columns=['Timestamp', 'Open', 'High',
'Low', 'Close', 'Volume'])
            df['Timestamp'] =
pd.to_datetime(df['Timestamp'], unit='ms')
            df.set_index('Timestamp',
inplace=True)
            buf = io.BytesIO()
            mc =
mpf.make_marketcolors(up='#00C48C',
down='#ff3366', inherit=True)
            s =
mpf.make_mpf_style(base_mpf_style='charles'
, marketcolors=mc, gridcolor='#222222',
facecolor='#0d0d0d')
            def _plot():

```

```

        mpf.plot(df, type='candle',
style=s, volume=True,
savefig=dict(fname=buf, dpi=115,
bbox_inches='tight'), figratio=(14,9))
        await
loop.run_in_executor(None, _plot)
        buf.seek(0)
        await sm.delete()
        await
u.message.reply_photo(photo=buf,
caption=f"
```

```

results = []
try:
    parsed = feedparser.parse(url)
    for entry in parsed.entries[:20]:
        title = entry.get("title",
        "").strip()
        title = re.sub(r'\s*-\s*[^-]
{3,40}$', '', title).strip()
        link = entry.get("link", "#")
        pub_date =
entry.get("published", "")[:16]
        summary_html =
entry.get("summary", "")
        img_url = None
        img_match = re.search(r'src=
["\'](https://[^"\']+\.
(?:jpg|jpeg|png|webp|gif))["\']',
summary_html, re.IGNORECASE)
        if img_match:
            img_url =
img_match.group(1)
            full_text = ""
            try:
                r = requests.get(link,
headers=G_HDR, timeout=8)
                if r.status_code == 200:
                    soup =
BeautifulSoup(r.text, "html.parser")
                    for tag in
soup(["script", "style", "nav", "footer", "head
er", "aside"]):
                        tag.decompose()
                    paragraphs =
[p.get_text().strip() for p in

```



```

soup.find_all('p') if
len(p.get_text().strip()) > 50]
        full_text = "
".join(paragraphs[:8])[:1800]
        if not img_url:
            og_img =
soup.find("meta", property="og:image")
            if og_img and
og_img.get("content"):
                img_url =
og_img["content"]
        except Exception:
            pass
        summary_text =
clean_html(summary_html)[:200] if not
full_text else full_text[:1800]
        if title:
            results.append({"title":
title[:200], "link": link, "summary":
summary_text, "image": img_url, "date":
pub_date})
        except Exception as e:
            logger.error(f"[fetch_google_news]
{e}")
    return results

def _format_news_body(summary: str) -> str:
    sentences = re.split(r'(?<=[.!?])\s+',
summary.strip())
    lines = []
    for s in sentences:
        s = s.strip()
        if len(s) > 20:
            lines.append(s)

```

```

        if len(lines) >= 18:
            break
        return "\n".join(lines) if lines else
summary[:800]

async def execute_news_flow(u: Update, c:
ContextTypes.DEFAULT_TYPE, feed_type: str,
label: str):
    if not u.message:
        return
    try:
        cid = u.effective_chat.id
        await safe_react(c.bot, cid,
u.message.message_id, "📰")
        sm = await
u.message.reply_text(f"📰 *Fetching
{label}...*",
parse_mode=ParseMode.MARKDOWN)
        loop = asyncio.get_running_loop()
        now = time.time()
        if news_cache[feed_type]["ts"] and
(now - news_cache[feed_type]["ts"]) < 300:
            items = news_cache[feed_type]
["data"]
        else:
            items = await
loop.run_in_executor(None,
fetch_google_news, feed_type)
            news_cache[feed_type]["ts"] =
now
            news_cache[feed_type]["data"] =
items
        if not items:
            await sm.edit_text("🐱 No news

```

```

found right now.")
    return
    await sm.delete()
    top = random.choice(items[:min(8,
len(items))])
    body =
_format_news_body(top["summary"])
    caption = (f"📰 *
{label.upper()}*\n—————\n\n"
              f"📌 *
{top['title']}*\n\n{body}\n\n—————
—————\n📅 {top['date']}")
    if len(caption) > 1020:
        caption = caption[:1017] +
        "...
    kb =
InlineKeyboardMarkup([[InlineKeyboardButton
("📖 Read Full Article",
url=top["link"])]])
    sent = False
    if top["image"]:
        try:
            await
u.message.reply_photo(photo=top["image"],
caption=caption,
parse_mode=ParseMode.MARKDOWN,
reply_markup=kb,
reply_to_message_id=u.message.message_id)
            sent = True
        except Exception:
            logger.warning("[news_img]
image send failed, falling back to text")
            if not sent:

```

```

        await
u.message.reply_text(caption,
parse_mode=ParseMode.MARKDOWN,
reply_markup=kb,
reply_to_message_id=u.message.message_id)
        bot_status["message_count"] += 1
    except Exception as e:
        logger.error(f"[execute_news_flow]
{e}")

async def qr_generate_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        parts =
u.message.text.split(maxsplit=1)
        if len(parts) < 2 or not
parts[1].strip():
            await u.message.reply_text("🐱
Usage: `/qr text here`")
            return
        payload = parts[1].strip()
        sm = await u.message.reply_text("■
*Generating QR Code...*",
parse_mode=ParseMode.MARKDOWN)
        loop = asyncio.get_running_loop()
        def _build():
            qr = qrcode.QRCode(version=1,
box_size=10, border=4)
            qr.add_data(payload)
            qr.make(fit=True)
            return
        qr.make_image(fill_color="black",

```

```

back_color="white")
    img = await
loop.run_in_executor(None, _build)
    bio = io.BytesIO()
    img.save(bio, "PNG")
    bio.seek(0)
    await sm.delete()
    await
u.message.reply_photo(photo=bio,
caption="📷 *QR Code Generated.*\n🐾 _Via  
Beluga Tools._")
    except Exception as e:
        logger.error(f"[qr_generate] {e}")

async def qr_scan_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message or not
u.message.reply_to_message or not
u.message.reply_to_message.photo:
        await u.message.reply_text("🐱
Reply to an image with `/scanqr`.")
        return
    try:
        sm = await u.message.reply_text("🟩
*Scanning QR Code...*",
parse_mode=ParseMode.MARKDOWN)
        photo =
u.message.reply_to_message.photo[-1]
        file_obj = await
c.bot.get_file(photo.file_id)
        buf = io.BytesIO()
        await
file_obj.download_to_memory(buf)
        buf.seek(0)

```

```

        loop = asyncio.get_running_loop()
        def _decode():
            arr =
np.frombuffer(buf.getvalue(),
dtype=np.uint8)
            img = cv2.imdecode(arr,
cv2.IMREAD_COLOR)
            detector = cv2.QRCodeDetector()
            val, _, _ =
detector.detectAndDecode(img)
            return val
            decoded_text = await
loop.run_in_executor(None, _decode)
            if decoded_text:
                await sm.edit_text(f"🤖
*Decoded:*\\n``\\n{decoded_text}\\n``",
parse_mode=ParseMode.MARKDOWN)
            else:
                await sm.edit_text("🐱 QR data
unreadable.")
        except Exception as e:
            logger.error(f"[qr_scan] {e}")

async def img_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE, action: str):
    if not u.message or not
u.message.reply_to_message or not
u.message.reply_to_message.photo:
        await u.message.reply_text("🐱
Reply to a photo.")
        return
    try:
        sm = await u.message.reply_text("📦
*Processing image...*",

```

```

parse_mode=ParseMode.MARKDOWN)
    p =
u.message.reply_to_message.photo[-1]
    f = await c.bot.get_file(p.file_id)
    b = io.BytesIO()
    await f.download_to_memory(b)
    b.seek(0)
    loop = asyncio.get_running_loop()
    if action == "info":
        im = Image.open(b)
        await sm.edit_text(
            f" *Image
Report*\n-----\n
*Resolution:* `{im.size[0]} x {im.size[1]}
pixels`\n *Color Mode:* `{im.mode}`\n
*Size:* `{p.file_size / 1024:.2f}
KB`\n-----",

parse_mode=ParseMode.MARKDOWN
    )
    elif action == "resize":
        def _scale():
            im = Image.open(b)
            out = im.resize((512, 512),
Image.Resampling.LANCZOS)
            out_b = io.BytesIO();
            out.save(out_b, "PNG"); out_b.seek(0)
            return out_b
        res_b = await
loop.run_in_executor(None, _scale)
        await sm.delete()
        await
u.message.reply_photo(photo=res_b,
caption=" *Resized to 512x512.*")

```

```

        elif action == "compress":
            def _crunch():
                im = Image.open(b)
                out_b = io.BytesIO();
            im.save(out_b, "JPEG", quality=22);
            out_b.seek(0)

            return out_b
            res_b = await
        loop.run_in_executor(None, _crunch)
            await sm.delete()
            await
        u.message.reply_photo(photo=res_b,
        caption="📁 *Compressed.*")
        except Exception as e:
            logger.error(f"[img_handler] {e}")

def _wrap_text(draw, text: str, font,
max_width: int) -> list:
    words = text.split()
    lines, current = [], ""
    for word in words:
        test = f"{current} {word}".strip()
    if current else word
        try:
            bbox = draw.textbbox((0, 0),
            test, font=font)
            test_w = bbox[2] - bbox[0]
        except Exception:
            test_w = len(test) * (font.size
            // 2)
        if test_w <= max_width:
            current = test
        else:
            if current:

```



```

        lines.append(current)
        current = word
    else:
        lines.append(word)
        current = ""
    if current:
        lines.append(current)
    return lines if lines else [text]

def _apply_watermark(buf: io.BytesIO,
wm_text: str, font_size: int, color_key:
str, style_name: str) -> io.BytesIO:
    im = Image.open(buf).convert("RGBA")
    img_w, img_h = im.size
    rgba = VIBGYOR_COLORS.get(color_key,
(255, 255, 255, 220))
    style_key = WM_STYLES.get(style_name,
"normal")

    margin_x = max(10, int(img_w * 0.04))
    max_text_width = img_w - 2 * margin_x

    chosen_font, chosen_lines = None, None
    for try_size in range(font_size, 11,
-2):
        font = load_font(style_key,
try_size)
        tmp_draw =
ImageDraw.Draw(Image.new("RGBA", (img_w,
img_h)))
        lines = _wrap_text(tmp_draw,
wm_text, font, max_text_width)
        try:
            lb = tmp_draw.textbbox((0, 0),

```

```

"Ay", font=font)
        line_h = (lb[3] - lb[1]) +
int(try_size * 0.25)
        except Exception:
            line_h = try_size + 4
            total_h = line_h * len(lines)
            if total_h <= img_h * 0.4:
                chosen_font, chosen_lines =
font, lines
                break

        if not chosen_font:
            chosen_font = load_font(style_key,
12)

            tmp_draw =
ImageDraw.Draw(Image.new("RGBA", (img_w,
img_h)))
            chosen_lines = _wrap_text(tmp_draw,
wm_text, chosen_font, max_text_width)

            txt_layer = Image.new("RGBA", im.size,
(0, 0, 0, 0))
            draw = ImageDraw.Draw(txt_layer)
            try:
                lb = draw.textbbox((0, 0), "Ay",
font=chosen_font)
                line_h = (lb[3] - lb[1]) +
int(chosen_font.size * 0.25)
            except Exception:
                line_h = chosen_font.size + 4

            total_text_h = line_h *
len(chosen_lines)
            block_top = img_h - total_text_h - 20

```

```

        shadow_color = (0, 0, 0, 140)
        shadow_offset = max(1, chosen_font.size
// 20)

        for i, line in enumerate(chosen_lines):
            try:
                lb2 = draw.textbbox((0, 0),
line, font=chosen_font)
                lw = lb2[2] - lb2[0]
            except Exception:
                lw = len(line) *
(chosen_font.size // 2)
                x = (img_w - lw) // 2
                y = block_top + i * line_h
                draw.text((x + shadow_offset, y +
shadow_offset), line, font=chosen_font,
fill=shadow_color)
                draw.text((x, y), line,
font=chosen_font, fill=rgba)

        combined = Image.alpha_composite(im,
txt_layer)
        out_b = io.BytesIO()
        combined.convert("RGB").save(out_b,
"JPEG", quality=92)
        out_b.seek(0)
        return out_b

    async def watermark_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
        if not u.message:
            return
        if not u.message.reply_to_message or
not u.message.reply_to_message.photo:

```

```

        await u.message.reply_text("🐱
Reply to a photo with `/watermark your
text`.")

        return
    parts =
u.message.text.split(maxsplit=1)
    wm_text = parts[1].strip() if
len(parts) > 1 else "BELUGAPY"
    uid = u.effective_user.id
    cid = u.effective_chat.id
    photo =
u.message.reply_to_message.photo[-1]
    wm_sessions[uid] = {"text": wm_text,
"file_id": photo.file_id, "chat_id": cid,
"step": "font_size"}
    sizes = [16, 24, 32, 40, 52, 64, 80,
96]
    rows, row = [], []
    for sz in sizes:

row.append(InlineKeyboardButton(str(sz),
callback_data=f"wm:size:{uid}:{sz}"))
        if len(row) == 4:
            rows.append(row); row = []
    if row:
        rows.append(row)
    await u.message.reply_text(f"✎
*Watermark:* `{wm_text}`\n\nStep 1 –
Choose *font size:*",
parse_mode=ParseMode.MARKDOWN,
reply_markup=InlineKeyboardMarkup(rows))

def _build_color_keyboard(uid: int) ->
InlineKeyboardMarkup:

```

```

        rows, row = [], []
        for label in VIBGYOR_COLORS:

            row.append(InlineKeyboardButton(label,
            callback_data=f"wm:color:{uid}:{label}"))
            if len(row) == 3:
                rows.append(row); row = []
        if row:
            rows.append(row)
        return InlineKeyboardMarkup(rows)

def _build_style_keyboard(uid: int) ->
InlineKeyboardMarkup:
    return
    InlineKeyboardMarkup([[InlineKeyboardButton
    (s, callback_data=f"wm:style:{uid}:{s}")]
    for s in WM_STYLES])

async def watermark_callback(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    if not q:
        return
    try:
        await q.answer()
        parts = q.data.split(":", 3)
        _, step, owner_uid_str, value =
parts
        owner_uid = int(owner_uid_str)
        if q.from_user.id != owner_uid:
            await q.answer("❌ This is not
your watermark session!", show_alert=True)
        return

```

```

    sess = wm_sessions.get(owner_uid)
    if not sess:
        await q.edit_message_text("🕒
Session expired. Use /watermark again.")
        return

    if step == "size":
        sess["font_size"] = int(value)
        sess["step"] = "color"
        await q.edit_message_text(f"✍️
*Watermark:* `{sess['text']}`\nFont size:
`{value}`\n\nStep 2 – Choose *text
color:*\n", parse_mode=ParseMode.MARKDOWN,
reply_markup=_build_color_keyboard(owner_uid))

    elif step == "color":
        sess["color"] = value
        sess["step"] = "style"
        await q.edit_message_text(f"✍️
*Watermark:* `{sess['text']}`\nSize:
`${sess['font_size']}` | Color:
`${value}`\n\nStep 3 – Choose *text
style:*\n", parse_mode=ParseMode.MARKDOWN,
reply_markup=_build_style_keyboard(owner_uid))

    elif step == "style":
        sess["style"] = value
        await q.edit_message_text("⚙️
*Applying watermark...*\n",
parse_mode=ParseMode.MARKDOWN)
        try:
            file_obj = await

```

```

context.bot.get_file(sess["file_id"])
        buf = io.BytesIO()
        await
file_obj.download_to_memory(buf)
        buf.seek(0)
        font_size =
sess.get("font_size", 40)
        color_key =
sess.get("color", "🕒 White")
        style_name =
sess.get("style", "Normal")
        wm_text = sess["text"]
        loop =
asyncio.get_running_loop()
        res_b = await
loop.run_in_executor(None,
    _apply_watermark, buf, wm_text, font_size,
    color_key, style_name)
        await
context.bot.send_photo(

    chat_id=sess["chat_id"], photo=res_b,
        caption=f"🛡️ *Watermark
Applied!*\\nText: `{wm_text}` | Size:
`{font_size}` | Color: `{color_key}` |
Style: `{style_name}`",

    parse_mode=ParseMode.MARKDOWN
        )
        wm_sessions.pop(owner_uid,
None)
    except Exception as e:
        logger.error(f"[wm_apply]
{e}")

```

```

        await
q.edit_message_text(f"🐱 Error: `{str(e)
[:80]}`", parse_mode=ParseMode.MARKDOWN)
    except Exception as e:
        logger.error(f"[wm_callback] {e}")

def quiz_on_cooldown(cid: str, question:
str) -> bool:
    return time.time() <
quiz_cooldown.get(cid,
{}).get(q_hash(question), 0)

def mark_quiz(cid: str, question: str):
    quiz_cooldown.setdefault(cid, {})
[q_hash(question)] = time.time() + 3600

async def gen_quiz(topic: str, cid: str) ->
Optional[dict]:
    for _ in range(2):
        try:
            raw = await ai("Trivia master.
Output ONLY raw JSON, no markdown.",
                            f"Topic:
'{topic}'. Generate 1 MC question.\n"

'{"question": "...", "options":
["A", "B", "C", "D"], "correct_index": 0, "fun_fa
ct": "..."}',

                                "", max_tok=200)
            if not raw:
                continue
            m = re.search(r"\{[\s\S]+\}",
raw)
            if not m:

```



```

        continue
        d = json.loads(m.group(0))
        q = str(d.get("question",
""))
.strip()
        opts = d.get("options", [])
        idx =
int(d.get("correct_index", 0))
        fact = str(d.get("fun_fact",
"Meow!"))
.strip()
        if not q or len(opts) != 4 or
not (0 <= idx <= 3):
            continue
        if quiz_on_cooldown(cid, q):
            continue
        return {"question": q,
"options": opts, "correct_index": idx,
"fun_fact": fact}
    except Exception:
        pass
    return None

async def quiz_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        parts =
u.message.text.split(maxsplit=1)
        topic = (parts[1].strip() if
len(parts) > 1 else None) or
random.choice(QUIZ_TOPICS)
        cid, cid_i =
str(u.effective_chat.id),
u.effective_chat.id

```

```

        await safe_react(c.bot, cid_i,
u.message.message_id, "💡")
        sm = await u.message.reply_text("🤖
*Generating quiz...*",
parse_mode=ParseMode.MARKDOWN)
        qdata = await gen_quiz(topic, cid)
        try:
            await sm.delete()
        except Exception:
            pass
        if qdata:
            mark_quiz(cid,
qdata["question"])
            try:
                pm = await c.bot.send_poll(
                    chat_id=cid_i,
question=f"🐱 {qdata['question'][:255]}",
                    options=[str(o)[:100]
for o in qdata["options"]],
                    type="quiz",
correct_option_id=qdata["correct_index"],
                    is_anonymous=False,
explanation=qdata["fun_fact"][:200]
                )
                active_polls[pm.poll.id] =
{"chat_id": cid_i, "correct_index":
qdata["correct_index"]}
                bot_status["message_count"]
+= 1

            return
        except Exception:
            pass
        fb = random.choice(FALLBACK_QS)
        mark_quiz(cid, fb["q"])

```

```

        pm = await
c.bot.send_poll(chat_id=cid_i,
question=f"🐱 {fb['q']}",
options=fb["opts"], type="quiz",

correct_option_id=fb["ans"],
is_anonymous=False, explanation=fb["fact"])
        active_polls[pm.poll.id] =
{"chat_id": cid_i, "correct_index":
fb["ans"]}
        bot_status["message_count"] += 1
    except Exception as e:
        logger.error(f"[quiz] {e}")

async def poll_answer_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    try:
        ans = u.poll_answer
        if not ans:
            return
        info =
active_polls.get(ans.poll_id)
        if not info or not ans.option_ids
or ans.option_ids[0] !=
info["correct_index"]:
            return
        cid, uid = str(info["chat_id"]),
str(ans.user.id)
        name = (ans.user.first_name or "?")
[:30]
        bump_score(cid, uid, name, +10)
        await
sync_leaderboard_chat_to_mongo(cid)
    except Exception:

```

```

pass

async def lb_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        cid = str(u.effective_chat.id)
        local_scores = db.get("scores",
        {}).get(cid, {})
        lb = sorted(local_scores.values(),
        key=lambda x: x.get("score", 0),
        reverse=True)
        seen_ids = set()
        clean_lb = [e for e in lb if
        e.get("user_id") not in seen_ids and not
        seen_ids.add(e.get("user_id"))]

        lw = db.get("weekly", {}).get(cid,
        {})
        lines = []
        if lw and lw.get("top3"):
            lines.append("🏆 *LAST WEEK
CHAMPIONS* 🏆\n")
            lines.extend([f"{MEDALS[i]}
{e.get('name','?')[:18]} -
{e.get('score',0):,} pts" for i, e in
            enumerate(lw["top3"])])

        lines.append("\n—————\n"
        )

        lines +=
        ["⌈—————⌋",

```

```

"""🏆 *CURRENT LEADERBOARD* 🏆",
"""|-----|\n"]

    if not clean_lb:
        lines.append("No scores yet!
Play some games 🎮")
    else:
        for i, e in
enumerate(clean_lb[:10]):
            m = MEDALS[i] if i <
len(MEDALS) else f"{i+1}."
            lines.append(f"{m}
`{e.get('name', 'Unknown')[:18]:<18}`
`{e.get('score', 0):>6,} pts`)")
            lines +=
["\n-----", "🏆 +10
quiz/ttt · +700 mine · +50 gm"]
            text = "\n".join(lines)

        try:
            await
u.message.reply_photo(photo=LB_IMAGE_URL,
caption=text,
parse_mode=ParseMode.MARKDOWN)
        except Exception:
            await
u.message.reply_text(text,
parse_mode=ParseMode.MARKDOWN)
            bot_status["message_count"] += 1
        except Exception as e:
            logger.error(f"[lb] {e}")

async def nw_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    """

```

New Week reset:

1. Read CURRENT in-memory scores for this chat.

2. Compute top 3 -> store as this chat's "weekly" champions (kept, not reset).

3. Wipe this chat's scores.

4. Push both changes to MongoDB immediately.

```
"""
if not u.message:
    return
try:
    if not is_owner(u.effective_user.id
if u.effective_user else 0):
        await u.message.reply_text("🚫
Owner only.")
        return
    cid = str(u.effective_chat.id)
    lb = sorted(db.get("scores",
{}).get(cid, {}).values(), key=lambda x:
x.get("score", 0), reverse=True)
    seen_ids = set()
    clean_lb = [e for e in lb if
e.get("user_id") not in seen_ids and not
seen_ids.add(e.get("user_id"))]
    top3 = [{"name": e.get("name",
"?"), "score": e.get("score", 0)} for e in
clean_lb[:3]]
    wk_label =
datetime.now().strftime("%d %b %Y")
    db.setdefault("weekly", {})[cid] =
{"top3": top3, "week_label": wk_label}
    db["scores"][cid] = {}
```

```

        await sync_weekly_to_mongo(cid)
        await
sync_leaderboard_chat_to_mongo(cid)

        announce = ["🏆🎉 *NEW WEEK!* 🎉🏆", f"\n_Week: {wk_label}_\n", "👑",
        *Champions:*"]
        announce.extend([f"{MEDALS[i]} *{e['name']}* - {e['score']:,} pts" for i, e
        in enumerate(top3)])
        announce.extend(["\n🔄 *All scores reset!*", "🚀 _New battle begins!_"])
        await
u.message.reply_text("\n".join(announce),
        parse_mode=ParseMode.MARKDOWN)
        bot_status["message_count"] += 1
    except Exception as e:
        logger.error(f"[nw] {e}")

async def pump_dump_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        if not is_owner(u.effective_user.id
        if u.effective_user else 0):
            await u.message.reply_text("🚫
Owner only.")
            return
        if not u.message.reply_to_message
or not
u.message.reply_to_message.from_user:
            await u.message.reply_text("⚠️
Reply to a user.")

```

```

        return
        parts = u.message.text.split()
        if len(parts) < 2 or not
parts[1].isdigit():
            await u.message.reply_text("⚠️
Usage: `/pump 100`")
            return
        amount = int(parts[1])
        cmd =
parts[0].lstrip("/").lower().split("@")[0]
        delta = +amount if cmd == "pump"
else -amount
        target, cid =
u.message.reply_to_message.from_user,
str(u.effective_chat.id)
        new_sc = bump_score(cid,
str(target.id), (target.first_name or
"User")[:30], delta)
        await
sync_leaderboard_chat_to_mongo(cid)
        emoji = "🚀" if cmd == "pump" else
"📉"
        sign = "+" if delta > 0 else ""
        await u.message.reply_text(
            f"{emoji} *{'PUMP' if
cmd=='pump' else 'DUMP'}*\n\n👤 *
{target.first_name}*\n{'📈' if delta>0 else
'📉'} {sign}{amount:},} pts\n💰 New total: *
{new_sc:},} pts*",
            parse_mode=ParseMode.MARKDOWN
        )
    except Exception as e:
        logger.error(f"[pump_dump] {e}")

```



```

async def fun_dispatcher(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        cid = str(u.effective_chat.id)
        db.setdefault("seen",
{}).setdefault(cid, {})
        cmd =
u.message.text.lower().split()
[0].lstrip("/").split("@")[0]
        active_users = list(db.get("seen",
{}).get(cid, {}).values())
        if len(active_users) < (2 if cmd ==
"couple" else 1) and OWNER_ID:
            active_users.append({"id":
OWNER_ID, "un": "Owner", "n": "Owner"})
        if len(active_users) < (2 if cmd ==
"couple" else 1):
            await u.message.reply_text("🐱
Need more group members!")
            return
        day = datetime.now().strftime("%y-
%m-%d")
        lk = f"{cid}:{cmd}:{day}"
        async with fun_cache_lock:
            cached =
fun_db.get("gay_couple_log", {}).get(lk)
            if cached and
cached.get("date") == day:
                await
u.message.reply_text(cached["result"],
parse_mode=ParseMode.MARKDOWN)
            return

```

```

        if cmd == "couple":
            m = random.sample(active_users,
min(2, len(active_users)))
            res = f"💕 *{m[0]['n']}* 💕 *
{m[1]['n']}*\n100% compatible!" if len(m)
== 2 else f"💕 *{m[0]['n']}* needs a
partner! 💔"
            else:
                m =
[random.choice(active_users)]
                res = f"🌈 *{m[0]['n']}* is
today's rainbow! 🌈"
                fun_db.setdefault("gay_couple_log",
{})[lk] = {"date": day, "result": res,
"users": [p.get("id") for p in m]}
                await u.message.reply_text(res,
parse_mode=ParseMode.MARKDOWN)
            except Exception as e:
                logger.error(f"[fun_dispatcher]
{e}")

def _build_gm_caption(users: list,
date_str: str) -> str:
    display_users = users[-15:] if
len(users) > 15 else users
    lines = ["📷 *DAILY ATTENDANCE*\n", "🙄
Mark attendance!\n", f"📅 {date_str} | 👥
{len(users)}\n",
"—————\n"]
    if len(users) > 15:
        lines.append(f"... +{len(users)-15}
more...\n")
    for i, user in enumerate(display_users,
1):

```

```

        lines.append(f"{i}. {user['name']}
        • {user['time']}}")
        lines +=
        ["\n—————\n", "🔥 +50
        pts for check-in"]
        return "\n".join(lines)

    async def gm_handler(u: Update, c:
    ContextTypes.DEFAULT_TYPE):
        if not u.message:
            return
        try:
            if not is_owner(u.effective_user.id
            if u.effective_user else 0):
                await u.message.reply_text("🚫
            Owner only.")
                return
            cid = str(u.effective_chat.id)
            date_str =
            datetime.now().strftime("%d %b %Y")
            msg = None
            try:
                msg = await
            u.message.reply_photo(
                photo=GM_IMAGE_URL,
            caption=_build_gm_caption([], date_str),

            reply_markup=InlineKeyboardMarkup([[InlineK
            eyboardButton("GM 🤔",
            callback_data=f"gm:attend:{cid}")]])
            )
        except Exception:
            msg = await
            u.message.reply_text(

```

```

        text=_build_gm_caption([],
date_str),

reply_markup=InlineKeyboardMarkup([[InlineK
eyboardButton("GM 🙄",
callback_data=f"gm:attend:{cid}"))]])
    )
    if msg:
        gm_tracker[cid] =
(msg.message_id, [], date_str)
        gm_msg_lock[cid] =
asyncio.Lock()
    except Exception as e:
        logger.error(f"[gm] {e}")

async def gm_callback(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    if not q:
        return
    try:
        parts = q.data.split(":")
        cid = parts[2]
        gm_msg_lock.setdefault(cid,
asyncio.Lock())
        async with gm_msg_lock[cid]:
            if cid not in gm_tracker:
                await q.answer("🕒
Expired")
            return
        msg_id, users, date_str =
gm_tracker[cid]
        user, user_id = q.from_user,
str(q.from_user.id)

```

```

        if any(uu.get("id") == user_id
for uu in users):
            await q.answer("✅ Already
marked")
            return
        u_name = (user.first_name or
"User")[:20]
        utime =
datetime.now().strftime("%H:%M")
        users.append({"id": user_id,
"name": u_name, "time": utime})
        gm_tracker[cid] = (msg_id,
users, date_str)

bump_score(str(q.message.chat_id), user_id,
u_name, +50)

        await
sync_leaderboard_chat_to_mongo(str(q.messag
e.chat_id))
        try:
            new_cap =
_build_gm_caption(users, date_str)
            kb =
InlineKeyboardMarkup([[InlineKeyboardButton
("GM 🥰", callback_data=f"gm:attend:
{cid}")]])

            if q.message.photo:
                await
context.bot.edit_message_caption(chat_id=q.
message.chat_id, message_id=msg_id,
caption=new_cap, reply_markup=kb,
parse_mode=ParseMode.MARKDOWN)
            else:
                await

```

```

context.bot.edit_message_text(chat_id=q.message.chat_id, message_id=msg_id,
text=new_cap, reply_markup=kb,
parse_mode=ParseMode.MARKDOWN)
        await q.answer(f"✅ +50
pts, {u_name}!")
    except Exception:
        await q.answer("✅
Marked!")
    except Exception as e:
        logger.error(f"[gm_callback] {e}")

TTT_EMPTY, TTT_X, TTT_O = " ", "X", "O"
WINS = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),
(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def ttt_check_winner(board):
    for a, b, cc in WINS:
        if board[a] == board[b] ==
board[cc] and board[a] != TTT_EMPTY:
            return board[a]
    return None

def ttt_is_draw(board):
    return all(c != TTT_EMPTY for c in
board) and not ttt_check_winner(board)

def _minimax(board, is_max, alpha, beta):
    w = ttt_check_winner(board)
    if w == TTT_O: return 10
    if w == TTT_X: return -10
    if all(c != TTT_EMPTY for c in board):
return 0
    best = -1000 if is_max else 1000

```

```

        for i in range(9):
            if board[i] != TTT_EMPTY:
                continue
            board[i] = TTT_0 if is_max else
TTT_X
            score = _minimax(board, not is_max,
alpha, beta)
            board[i] = TTT_EMPTY
            if is_max:
                best = max(best, score); alpha
= max(alpha, best)
            else:
                best = min(best, score); beta =
min(beta, best)
            if beta <= alpha:
                break
        return best

def ttt_bot_move(board):
    best_score, best_move = -1000, -1
    for i in range(9):
        if board[i] != TTT_EMPTY:
            continue
        board[i] = TTT_0
        score = _minimax(board, False,
-1000, 1000)
        board[i] = TTT_EMPTY
        if score > best_score:
            best_score, best_move = score,
i
    return best_move

def ttt_build_keyboard(board,
disabled=False):

```

```

rows = []
for row in range(3):
    r = []
    for col in range(3):
        idx = row * 3 + col
        cb = f"ttt:noop:{idx}" if
(board[idx] != TTT_EMPTY or disabled) else
f"ttt:move:{idx}"

r.append(InlineKeyboardButton(board[idx],
callback_data=cb))
    rows.append(r)
return InlineKeyboardMarkup(rows)

def ttt_build_text(g):
    gkey = f"{g['chat_id']}":
    {g.get('msg_id', '')}"
    rem = game_timers.get(gkey,
    {}).get("remaining", 300)
    tsec = f"{rem//60:02d}:{rem%60:02d}"
    board_str = "\n".join(["
    ".join(g["board"][r*3+col] for col in
    range(3)) for r in range(3)])
    status = g.get("status", "playing")
    if status == "playing":
        sl = f"🎯 *{g['x_name']} if
g['turn'] == 'X' else g['o_name']}'s turn*
{'❌' if g['turn'] == 'X' else '🔴'} 🕒
`{tsec}`"
        elif status == "timeout":
            sl = f"🕒 *Time up!*🏆 *
{g.get('winner_name', '')}* wins!"
        elif status == "draw":
            sl = "👉 *Draw!*"

```



```

else:
    sl = f"🏆 *
{g.get('winner_name', '')}* wins!"
    return f"🎮 *TIC TAC
TOE*\n-----\n❌ {g['x_name']}
vs 🟠
{g['o_name']}\n-----\n\n{board_
str}\n\n-----\n{sl}"

async def cleanup_expired_games():
    while True:
        await asyncio.sleep(60)
        now = time.time()
        for gkey in list(ttt_games.keys()):
            g = ttt_games[gkey]
            if now - g.get("created", now)
> 300:
                for uid in
[str(g.get("x_id", "")), str(g.get("o_id",
""))]:
                    user_in_game.pop(uid,
None)
                    game_timers.pop(gkey, None)
                    del ttt_games[gkey]

async def run_game_timer(c, gkey):
    try:
        while True:
            await asyncio.sleep(5)
            g, td = ttt_games.get(gkey),
game_timers.get(gkey)
            if not g or not td or
g.get("status") != "playing":
                return

```

```

        td["remaining"] = max(0,
td["remaining"] - 5)
        cid, msg_id = g.get("chat_id"),
g.get("msg_id")
        if not msg_id:
            return
        if td["remaining"] <= 0:
            g["status"] = "timeout"
            g["winner_name"] =
(g["o_name"] if g["turn"] == "X" else
g["x_name"])
            try:
                await
c.bot.edit_message_text(chat_id=cid,
message_id=msg_id, text=ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(g["board"],
disabled=True))
            except Exception:
                pass
            for uid in
[str(g.get("x_id", "")), str(g.get("o_id",
""))]:
                user_in_game.pop(uid,
None)
                game_timers.pop(gkey,
None); ttt_games.pop(gkey, None)
            return
        try:
            await
c.bot.edit_message_text(chat_id=cid,
message_id=msg_id, text=ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(g["board"]))

```

```

)

        except Exception:
            pass
    except asyncio.CancelledError:
        pass

def player_busy(uid):
    gkey = user_in_game.get(uid)
    if not gkey:
        return False
    if gkey in ttt_games:
        return True
    user_in_game.pop(uid, None)
    return False

async def tictac_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        ua, cid, uid_a = u.effective_user,
u.effective_chat.id,
str(u.effective_user.id)
        name_a = (ua.first_name or
"Player")[:20]
        vs_bot, user_b_id, name_b = True,
None, "🤖 Bot"
        if u.message.reply_to_message and
u.message.reply_to_message.from_user:
            rb =
u.message.reply_to_message.from_user
            if not rb.is_bot:
                vs_bot, user_b_id, name_b =
False, rb.id, (rb.first_name or "Player2")

```

```

[:20]
        if player_busy(str(rb.id)):
            await
u.message.reply_text("⚠ Player in game!");
return
        if player_busy(uid_a):
            await u.message.reply_text("⚠
You're already in a game!"); return
        board = [TTT_EMPTY] * 9
        g = {"board": board, "turn": "X",
"x_id": ua.id, "x_name": name_a, "o_id":
user_b_id if not vs_bot else -1,
        "o_name": name_b, "vs_bot":
vs_bot, "status": "playing", "created":
time.time(), "chat_id": cid, "msg_id":
None}

        msg = await
u.message.reply_text(ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(board))
        g["msg_id"] = msg.message_id
        gkey = game_key(msg.message_id,
cid)

        ttt_games[gkey] = g
        game_timers[gkey] = {"remaining":
300}

        user_in_game[uid_a] = gkey
        if not vs_bot:
            user_in_game[str(user_b_id)] =
gkey

        asyncio.create_task(run_game_timer(c,
gkey))
    except Exception as e:

```

```

        logger.error(f"[tictac] {e}")

    async def ttt_callback(update: Update,
context: ContextTypes.DEFAULT_TYPE):
        q = update.callback_query
        if not q:
            return
        try:
            try: await q.answer()
            except Exception: pass
            parts = q.data.split(":")
            action, idx = parts[1],
int(parts[2]) if len(parts) > 2 else -1
            cid, mid = q.message.chat_id,
q.message.message_id
            gkey = game_key(mid, cid)
            g = ttt_games.get(gkey)
            if not g or g["status"] !=
"playing" or action == "noop":
                return
            uid = str(q.from_user.id)
            if g["turn"] == "X" and uid !=
str(g["x_id"]): return
            if g["turn"] == "O" and not
g["vs_bot"] and uid != str(g["o_id"]):
return
            board = g["board"]
            if idx < 0 or idx >= 9 or
board[idx] != TTT_EMPTY:
                return
            if gkey in game_timers:
                game_timers[gkey]["remaining"]
= 300
            board[idx] = TTT_X if g["turn"] ==

```

```

"X" else TTT_0
    ws = ttt_check_winner(board)
    if ws:
        g["status"], g["winner_name"] =
"win", (g["x_name"] if ws == TTT_X else
g["o_name"])
        winner_uid = str(g["x_id"]) if
ws == TTT_X else str(g["o_id"]) if not
g["vs_bot"] else -1)
        if winner_uid and winner_uid !=
"-1":
            bump_score(str(cid),
winner_uid, g["winner_name"], +10)
            await
sync_leaderboard_chat_to_mongo(str(cid))
            try: await
q.edit_message_text(ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(board,
disabled=True))
            except Exception: pass
        for uid in [str(g["x_id"]),
str(g["o_id"])]: user_in_game.pop(uid,
None)
        game_timers.pop(gkey, None);
ttt_games.pop(gkey, None); return
        if ttt_is_draw(board):
            g["status"] = "draw"
            try: await
q.edit_message_text(ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(board,
disabled=True))
            except Exception: pass

```

```

        for uid in [str(g["x_id"]),
str(g["o_id"])]: user_in_game.pop(uid,
None)

        game_timers.pop(gkey, None);
ttr_games.pop(gkey, None); return
        g["turn"] = "0" if g["turn"] == "X"
else "X"
        if g["vs_bot"] and g["turn"] ==
"0":

            bi = ttt_bot_move(board)
            if bi >= 0:
                board[bi] = TTT_0
                ws2 =
ttt_check_winner(board)
                if ws2 or
ttt_is_draw(board):
                    g["status"] = "win" if
ws2 else "draw"
                    if ws2:
g["winner_name"] = (g["x_name"] if ws2 ==
TTT_X else g["o_name"])
                    try: await
q.edit_message_text(ttt_build_text(g),
parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(board,
disabled=True))
                    except Exception: pass

user_in_game.pop(str(g["x_id"]), None)
        game_timers.pop(gkey,
None); ttr_games.pop(gkey, None); return
        g["turn"] = "X"
        try: await
q.edit_message_text(ttt_build_text(g),

```

```

parse_mode=ParseMode.MARKDOWN,
reply_markup=ttt_build_keyboard(board))
    except Exception: pass
except Exception as e:
    logger.error(f"[ttt_cb] {e}")

def _mine_setup_keyboard(gkey):
    return InlineKeyboardMarkup([[
        InlineKeyboardButton("3 Mines",
callback_data=f"mine:set:{gkey}:3"),
        InlineKeyboardButton("4 Mines",
callback_data=f"mine:set:{gkey}:4"),
        InlineKeyboardButton("5 Mines",
callback_data=f"mine:set:{gkey}:5")
    ]])

def _mine_board_keyboard(gkey, state,
revealed, disabled=False):
    rows, r = [], []
    for i in range(6):
        if disabled or revealed[i]:
            label = "💣" if state[i] else
("✅" if revealed[i] else "❌")
        else:
            label = "📦"
            cb = f"mine:play:{gkey}:{i}" if not
disabled and not revealed[i] else
f"mine:noop:{gkey}:{i}"

    r.append(InlineKeyboardButton(label,
callback_data=cb))
    if len(r) == 3:
        rows.append(r); r = []
    if r:

```



```

        rows.append(r)
    return InlineKeyboardMarkup(rows)

def mine_build_text(g, rem):
    bombs, total_safe, opened = g["bombs"],
    6 - g["bombs"], sum(1 for x in
    g["revealed"] if x)
    if g.get("status") == "timeout": return
    "🕒 *Time Up!*\n\nLost *-5 pts*."
    elif g.get("status") == "lost": return
    "💣 *BOOM!*\n\nLost *-5 pts*."
    elif g.get("status") == "won": return
    f"🎉 *YOU WIN!*\n\nAll {total_safe} safe
    boxes found! Won *+700 pts*."
    else: return f"💣 *MINESWEEPER*\nMines:
    {bombs} | Safe: {opened}/{total_safe}\n⏱
    Time: `{rem}s`"

async def mine_handler(u: Update, c:
    ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    try:
        cid, uid =
        str(u.effective_chat.id),
        str(u.effective_user.id)
        now = time.time()
        m_stat =
        mine_play_stats.setdefault(uid, {"plays":
        0, "block_until": 0})
        if now < m_stat["block_until"]:
            rem_m = max(1,
            int((m_stat["block_until"] - now) // 60))
            await u.message.reply_text(f"⌚

```

```

*Cooldown!*\\nWait {rem_m} minutes.",
parse_mode=ParseMode.MARKDOWN); return
    m_stat["plays"] += 1
    if m_stat["plays"] > 20:
        m_stat["block_until"] = now +
3600; m_stat["plays"] = 0
        await u.message.reply_text("🔴
*Limit Hit!*\\n1-hour break.",
parse_mode=ParseMode.MARKDOWN); return
        gkey = f"{cid}_{uid}_{int(now)}"
        mine_games[gkey] = {"uid": uid,
"name": (u.effective_user.first_name or
"Player")[:20], "bombs": 0,
                                "state": [],
"revealed": [False]*6, "chat_id":
u.effective_chat.id, "msg_id": None,
"status": "setting"}
        msg = await
u.message.reply_photo(photo=MINE_IMAGE_URL,
caption="💣 *MINESWEEPER*\\n\\nChoose number
of mines:", parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_setup_keyboard(gkey))
        mine_games[gkey]["msg_id"] =
msg.message_id
    except Exception as e:
        logger.error(f"[mine] {e}")

async def run_mine_timer(c, gkey):
    try:
        while True:
            await asyncio.sleep(5)
            g, td = mine_games.get(gkey),
mine_timers.get(gkey)
            if not g or not td or

```

```

g.get("status") != "playing":
    return
    td["remaining"] = max(0,
td["remaining"] - 5)
    cid, msg_id = g.get("chat_id"),
g.get("msg_id")
    if not msg_id:
        return
    if td["remaining"] <= 0:
        g["status"] = "timeout"
        new_sc =
bump_score(str(cid), g["uid"], g["name"],
-5)

        await
sync_leaderboard_chat_to_mongo(str(cid))
        try:
            await
c.bot.edit_message_caption(chat_id=cid,
message_id=msg_id,
caption=mine_build_text(g, 0) +
f"\n\nBalance: *{new_sc:,} pts*",
parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
g["state"], g["revealed"], disabled=True))
        except Exception:
            pass
            mine_timers.pop(gkey,
None); mine_games.pop(gkey, None)
        return
    try:
        await
c.bot.edit_message_caption(chat_id=cid,
message_id=msg_id,
caption=mine_build_text(g,

```

```

td["remaining"])),
parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
g["state"], g["revealed"]))
    except Exception:
        pass
except asyncio.CancelledError:
    pass

async def mine_callback(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    if not q:
        return
    try:
        try: await q.answer()
        except Exception: pass
        parts = q.data.split(":")
        action, gkey, val = parts[1],
parts[2], int(parts[3])
        if gkey not in mine_games:
            return
        g = mine_games[gkey]
        if str(q.from_user.id) != g["uid"]:
            await q.answer("Not your
game!"); return
        if action == "noop":
            return
        if action == "set":
            if g.get("status") !=
"setting": return
            bombs = max(3, min(5, val))
            state = [True]*bombs + [False]*
(6-bombs)

```

```

        random.shuffle(state)
        g.update({"bombs": bombs,
"state": state, "status": "playing",
"revealed": [False]*6})
        mine_timers[gkey] =
{"remaining": 60}

asyncio.create_task(run_mine_timer(context,
gkey))

        try: await
q.edit_message_caption(caption=mine_build_t
ext(g, 60), parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
state, g["revealed"]))
        except Exception: pass
        elif action == "play":
            if g.get("status") != "playing"
or g["revealed"][val]:
                return
            is_bomb = g["state"][val]
            cid = str(q.message.chat_id)
            if is_bomb:
                g["status"] = "lost"
                mine_timers.pop(gkey, None)
                new_sc = bump_score(cid,
g["uid"], g["name"], -5)
                await
sync_leaderboard_chat_to_mongo(cid)
                try: await
q.edit_message_caption(caption=mine_build_t
ext(g, 0) + f"\n\nBalance: *{new_sc:},}
pts*", parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
g["state"], g["revealed"], disabled=True))

```

```

        except Exception: pass
        mine_games.pop(gkey, None)
    else:
        g["revealed"][val] = True
        total_safe = 6 - g["bombs"]
        opened_count = sum(1 for x
in g["revealed"] if x)
        if gkey in mine_timers:
            mine_timers[gkey]
["remaining"] = 60
            if opened_count >=
total_safe:
                g["status"] = "won"
                mine_timers.pop(gkey,
None)

                new_sc =
bump_score(cid, g["uid"], g["name"], +700)
                await
sync_leaderboard_chat_to_mongo(cid)
                try: await
q.edit_message_caption(caption=mine_build_t
ext(g, 0) + f"\n\nBalance: *{new_sc:},{
pts*\"", parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
g["state"], g["revealed"], disabled=True))
                except Exception: pass
                mine_games.pop(gkey,
None)

            else:
                rem =
mine_timers.get(gkey, {}).get("remaining",
60)

                try: await
q.edit_message_caption(caption=mine_build_t

```

```

ext(g, rem), parse_mode=ParseMode.MARKDOWN,
reply_markup=_mine_board_keyboard(gkey,
g["state"], g["revealed"]))
        except Exception: pass
    except Exception as e:
        logger.error(f"[mine_callback]
{e}")

def wiki_summary(query):
    """
    Uses Wikipedia's OFFICIAL REST v1 API
    (https://www.mediawiki.org/wiki/API:REST_AP
    I)
    instead of scraping the legacy
    action=query endpoint. Two calls:
        1. /search/page -> find the best-
        matching title for the query
        2. /page/{title}/summary -> official
        clean summary + thumbnail image
    Return shape kept identical to before
    so every caller (search_handler,
    bananalogic_handler, wiki_page_image)
    keeps working unchanged.
    """
    out = {"found": False, "title": "",
"url": "", "intro": "", "sections": [],
"image": None}
    try:
        sr = requests.get(

"https://en.wikipedia.org/w/rest.php/v1/sear
ch/page",
        params={"q": query, "limit":
1},

```

```

        headers=WIKI-UA, timeout=10,
    )
    if sr.status_code != 200:
        return out
    hits = sr.json().get("pages", [])
    if not hits:
        return out
    best_title = hits[0]["title"]
    best_key = hits[0].get("key",
best_title.replace(" ", "_"))

    summary_r = requests.get(

f"https://en.wikipedia.org/api/rest_v1/page
/summary/{urllib.parse.quote(best_key)}",
        headers=WIKI-UA, timeout=10,
    )
    if summary_r.status_code != 200:
        return out
    data = summary_r.json()
    intro = data.get("extract",
    "").strip()
    if not intro:
        return out
    page_url = data.get("content_urls",
    {}).get("desktop", {}).get("page") \
        or
f"https://en.wikipedia.org/wiki/{urllib.par
se.quote(best_key)}"
    thumb = data.get("thumbnail",
    {}).get("source") or
data.get("originalimage", {}).get("source")

    out.update({

```



```

        "found": True, "title":
best_title, "url": page_url,
        "intro": intro[:1500],
"sections": [], "image": thumb,
    })
    except Exception:
        pass
    return out

def google_search(query):
    out = {"found": False, "ai_answer": "",
"snippets": []}
    try:
        r =
requests.get(f"https://www.google.com/search?q=
{urllib.parse.quote_plus(query)}&num=8&hl=en", headers=G_HDR, timeout=10)
        if r.status_code != 200:
            return out
        html = r.text
        for pat in [r'data-
attrid="wa:/description"^(^>)*>[\s\S]
{0,200}?<span^(^>)*>(^<){40,800}', r'<div
class="BNeawe s3v9rd AP7Wnd">([\s\S]
{40,800}?)</div>']:
            m = re.search(pat, html,
re.DOTALL)
            if m:
                out["ai_answer"] =
clean_html(m.group(1))[:800]; break
        seen = set()
        for m in re.finditer(r'class="
[^"]*VwiC3b[^"]*"^(^>)*>([\s\S]{40,350}?)

```

```

</div', html, re.DOTALL):
    t = clean_html(m.group(1))
    if len(t) > 40 and t not in
seen:
        seen.add(t);
out["snippets"].append(t[:300])
    if len(out["snippets"]) >= 5:
        break
    out["found"] =
bool(out["ai_answer"] or out["snippets"])
    except Exception:
        pass
    return out

def google_image_search(query: str) ->
Optional[str]:
    """Scrapes a single relevant image URL
    from Google Images for the query. Best-
    effort, returns None on failure."""
    try:
        r = requests.get(

f"https://www.google.com/search?q=
{urllib.parse.quote_plus(query)}&tbm=isch",
        headers=G_HDR, timeout=10,
        )
        if r.status_code != 200:
            return None
        matches = re.findall(r'\["
(https://[^\"]+\.(?:jpg|jpeg|png|webp))"',
r.text)
        for url in matches:
            if "gstatic.com" not in url:
                return url

```

```

        return matches[0] if matches else
None
    except Exception:
        return None

async def web_summarise(query, wiki, goog,
system_prompt, max_tok=500):
    ctx = []
    if goog["ai_answer"]:
ctx.append(f"Google Featured Answer:
{goog['ai_answer']}")
    if goog["snippets"]: ctx.append("Web
snippets:\n" + "\n".join(f"- {s}" for s in
goog["snippets"]))
    if wiki["found"]:
ctx.append(f"Wikipedia
({wiki['title']}):\n{wiki['intro']}")
    if not ctx:
        return ""
    return await ai(system_prompt, f"User
question: {query}\n\nSearch
facts:\n{chr(10).join(ctx)
[:3000]}\n\nAnswer concisely.", "",
max_tok=max_tok)

def wiki_page_image(title: str) ->
Optional[str]:
    """
    Fetch the main thumbnail image URL for
a Wikipedia page via the raw
    MediaWiki `pageimages` prop
(wikipediaapi itself doesn't expose this).
    Returns None if the page has no image
or the request fails.

```

```

"""
    try:
        r = requests.get(

"https://en.wikipedia.org/w/api.php",
        params={
            "action": "query",
            "titles": title, "prop": "pageimages",
            "piprop": "original",
            "format": "json",
        },
        headers=WIKI-UA, timeout=10,
        )
        pages = r.json().get("query",
        {}).get("pages", {})
        for _pid, page in pages.items():
            original = page.get("original",
        {})

            if original.get("source"):
                return original["source"]
    except Exception:
        pass
    return None

def _is_url(text: str) -> bool:
    return bool(re.match(r"^https?://\S+$",
text.strip(), re.IGNORECASE))

async def _screenshot_website(u: Update, c:
ContextTypes.DEFAULT_TYPE, url: str):
    """
        /search <website link> – sends a
screenshot of the site instead of

```

running a text search. Uses WordPress's free mShots screenshot service (no API key needed). Crash-proof: falls back to a plain message with the link if the screenshot can't be generated or sent.

```
"""
    cid = u.effective_chat.id
    sm = await u.message.reply_text("📷
    *Taking a screenshot...*",
    parse_mode=ParseMode.MARKDOWN)
    shot_url =
    f"https://s.wordpress.com/mshots/v1/{urllib
    .parse.quote(url, safe='')}?w=1280&h=720"
    caption = f"📷 *Screenshot of:*\n{url}"
    try:
        await sm.delete()
    except Exception:
        pass
    sent = await send_photo_safe(c.bot,
    cid, shot_url, caption=caption,
    parse_mode=ParseMode.MARKDOWN)
    if not sent:
        try:
            await u.message.reply_text(f"😿
            Couldn't screenshot that site.\n{url}",
            reply_to_message_id=u.message.message_id)
        except Exception as e:
            logger.error(f"
            [_screenshot_website] {e}")
```

```
async def _run_search_and_reply(u: Update,
    c: ContextTypes.DEFAULT_TYPE, query: str):
```

```

"""Core search logic shared by /search
and the bare 'search X' / 'google X'
keyword trigger."""
    cid = u.effective_chat.id

    if _is_url(query):
        await _screenshot_website(u, c,
query)
        return

    await safe_react(c.bot, cid,
u.message.message_id, "🔍")
    sm = await u.message.reply_text("🔍
*Searching...*",
parse_mode=ParseMode.MARKDOWN)

    loop = asyncio.get_running_loop()
    wiki, goog = await asyncio.gather(
        loop.run_in_executor(None,
wiki_summary, query),
        loop.run_in_executor(None,
google_search, query),
    )

    image_url = wiki.get("image")
    if not image_url and wiki.get("found")
and wiki.get("title"):
        try:
            image_url = await
loop.run_in_executor(None, wiki_page_image,
wiki["title"])
        except Exception:
            image_url = None
    if not image_url:

```

```

        try:
            image_url = await
loop.run_in_executor(None,
google_image_search, query)
        except Exception:
            image_url = None

        summary = await web_summarise(
            query, wiki, goog,
            "Smart research assistant. Read the
given facts and write ONE tight summary "
            "in 60 to 100 words total, in clear
natural English. No headers, no bullet
points, "
            "just flowing prose that directly
answers what the person was searching
for.",
            max_tok=180,
        )

        if not summary and wiki.get("intro"):
            summary = wiki["intro"][:600]

        if not summary:
            try:
                await sm.edit_text("😸 No
results found. Try rephrasing?")
            except Exception:
                pass
            return

        try:
            await sm.delete()
        except Exception:

```

```

pass

caption = f"🔍 *{query}*\n\n{summary}"

if image_url:
    sent = await send_photo_safe(c.bot,
cid, image_url, caption=caption,
parse_mode=ParseMode.MARKDOWN)
    if sent:
        return

try:
    await u.message.reply_text(caption,
parse_mode=ParseMode.MARKDOWN,
reply_to_message_id=u.message.message_id)
except Exception as e:
    logger.error(f"
[_run_search_and_reply] Final text send
failed: {e}")

async def ss_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    """/ss <query> – sends the most
relevant Google Images result for a query.
For a website screenshot, use /search <url>
instead."""
    if not u.message:
        return
    parts =
u.message.text.split(maxsplit=1)
    if len(parts) < 2:
        await u.message.reply_text("🐱
Usage: `/ss query`",

```



```

parse_mode=ParseMode.MARKDOWN)
    return
    query = parts[1].strip()
    cid = u.effective_chat.id
    sm = await u.message.reply_text("🔍  

    *Finding image...*",
    parse_mode=ParseMode.MARKDOWN)
    loop = asyncio.get_running_loop()
    image_url = await
    loop.run_in_executor(None,
    google_image_search, query)
    try:
        await sm.delete()
    except Exception:
        pass
    if not image_url:
        await u.message.reply_text("😿 No  

    image found.")
    return
    sent = await send_photo_safe(c.bot,
    cid, image_url, caption=f"🔍 *{query}*",
    parse_mode=ParseMode.MARKDOWN)
    if not sent:
        await u.message.reply_text(f"😿  

    Couldn't send image for: {query}")

async def search_handler(u: Update, c:
    ContextTypes.DEFAULT_TYPE):
    """
    /search <query> – Wikipedia (official
    REST API) + web search, condensed
    into a 60-100 word AI summary with an
    image attached.

```

```

        /search <website link> – sends a
        screenshot of that site instead.
        """
        if not u.message:
            return
        parts =
u.message.text.split(maxsplit=1)
        if len(parts) < 2:
            await u.message.reply_text("🐱
Usage: `/search query` or `/search
https://example.com`,
parse_mode=ParseMode.MARKDOWN)
            return
            await _run_search_and_reply(u, c,
parts[1].strip())

async def bananalogic_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    parts =
u.message.text.split(maxsplit=1)
    if len(parts) < 2:
        await u.message.reply_text("🍌
Usage: `/bananalogic query`"); return
        query = parts[1].strip()
        cid = u.effective_chat.id
        await safe_react(c.bot, cid,
u.message.message_id, "🍌")
        sm = await u.message.reply_text("🍌
*BananaLogic searching...*",
parse_mode=ParseMode.MARKDOWN)
        loop = asyncio.get_running_loop()
        wiki, goog = await

```

```

asyncio.gather(loop.run_in_executor(None,
wiki_summary, query),
loop.run_in_executor(None, google_search,
query))
    answer = await web_summarise(query,
wiki, goog, BANANA_PROMPT, max_tok=600)

    if not answer:
        try:
            await sm.edit_text("🙌 No
response. Try again!")
        except Exception:
            pass
        return

    text = f"“ {query} ”\n\n{answer}\n\n
🐾 _via BananaLogic_"
    image_url = None
    if wiki.get("found") and
wiki.get("title"):
        try:
            image_url = await
loop.run_in_executor(None, wiki_page_image,
wiki["title"])
        except Exception:
            image_url = None

    try:
        await sm.delete()
    except Exception:
        pass

    if image_url:
        sent = await send_photo_safe(c.bot,

```

```

cid, image_url, caption=text,
parse_mode=ParseMode.MARKDOWN)
    if not sent:
        pass # send_photo_safe already
sent the text fallback
    else:
        try:
            await
u.message.reply_text(text,
parse_mode=ParseMode.MARKDOWN,
reply_to_message_id=u.message.message_id)
        except Exception as e:
            logger.error(f"
[bananalogic_handler] Final send failed:
{e}")

```

```

def _extract_youtube_video_id(query: str) -
> Optional[str]:
    """Pull a YouTube video ID out of any
common URL format, or return None if it's
not a URL."""
    patterns = [
        r"(?:youtube\.com/watch\?
v=|youtu\.be/|youtube\.com/shorts/|youtube\
.com/embed/)([A-Za-z0-9_-]{11})",
    ]
    for pat in patterns:
        m = re.search(pat, query)
        if m:
            return m.group(1)
    return None

```

```

def _format_yt_duration(iso_duration: str)
-> str:
    """Convert ISO 8601 duration (e.g.
    PT1H2M3S) to a readable H:MM:SS / M:SS
    string."""
    m = re.match(r"PT(?:\d+)H?(?:\d+)M?(?:\d+)S?", iso_duration or "")
    if not m:
        return "?"
    h, mi, s = (int(x) if x else 0 for x in
    m.groups())
    if h:
        return f"{h}:{mi:02d}:{s:02d}"
    return f"{mi}:{s:02d}"

def _format_yt_count(n: str) -> str:
    try:
        n = int(n)
    except (TypeError, ValueError):
        return "?"
    if n >= 1_000_000:
        return f"{n/1_000_000:.1f}M"
    if n >= 1_000:
        return f"{n/1_000:.1f}K"
    return str(n)

async def yt_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    """
    /yt <query or link> – looks up a
    YouTube video's metadata using the
    OFFICIAL YouTube Data API v3 (search +

```

```

videos endpoints). Sends back
    the thumbnail, title, channel,
duration, views, and a direct YouTube
    link. This does NOT download or
redistribute any video/audio content –
    only public metadata that the API is
designed to return.
"""

    if not u.message:
        return
    parts =
u.message.text.split(maxsplit=1)
    if len(parts) < 2:
        await u.message.reply_text("🎬
Usage: `/yt query or link`,
parse_mode=ParseMode.MARKDOWN)
        return
    query = parts[1].strip()

    if not YOUTUBE_API_KEY:
        await u.message.reply_text("🎬
YouTube lookup isn't configured right now
(missing API key).")
        return

    cid = u.effective_chat.id
    await safe_react(c.bot, cid,
u.message.message_id, "🎬")
    sm = await u.message.reply_text("🎬
*Looking that up...*",
parse_mode=ParseMode.MARKDOWN)

    loop = asyncio.get_running_loop()
    video_id =

```

```

_extract_youtube_video_id(query)

def _fetch_video_id_via_search():
    r = requests.get(

"https://www.googleapis.com/youtube/v3/search",
        params={"part": "snippet", "q":
query, "type": "video", "maxResults": 1,
"key": YOUTUBE_API_KEY},
        timeout=10,
    )
    if r.status_code != 200:
        return None
    items = r.json().get("items", [])
    if not items:
        return None
    return items[0]["id"]["videoId"]

def _fetch_video_details(vid: str):
    r = requests.get(

"https://www.googleapis.com/youtube/v3/videos",
        params={"part":
"snippet,contentDetails,statistics", "id":
vid, "key": YOUTUBE_API_KEY},
        timeout=10,
    )
    if r.status_code != 200:
        return None
    items = r.json().get("items", [])
    if not items:
        return None

```

```

        return items[0]

    try:
        if not video_id:
            video_id = await
loop.run_in_executor(None,
_fetch_video_id_via_search)
        if not video_id:
            try:
                await sm.edit_text("😸
Couldn't find that video.")
            except Exception:
                pass
            return

        details = await
loop.run_in_executor(None,
_fetch_video_details, video_id)
        if not details:
            try:
                await sm.edit_text("😸
Couldn't fetch video details right now.")
            except Exception:
                pass
            return

        snippet = details.get("snippet",
{})
        stats = details.get("statistics",
{})
        content =
details.get("contentDetails", {})

        title = snippet.get("title",

```



```

"Unknown title")
    channel =
snippet.get("channelTitle", "Unknown
channel")
    duration =
_format_yt_duration(content.get("duration",
""))
    views =
_format_yt_count(stats.get("viewCount",
"0"))
    likes =
_format_yt_count(stats.get("likeCount",
"0"))
    video_url =
f"https://youtu.be/{video_id}"
    thumb = (
        snippet.get("thumbnails",
{}).get("high", {}).get("url")
        or snippet.get("thumbnails",
{}).get("default", {}).get("url")
    )

    caption = (
        f"🎬 *{title}*\n\n"
        f"📺 {channel}\n"
        f"⌚ {duration} • 👁 {views}
views • 👍 {likes}\n\n"
        f"🔗 [Watch on YouTube]
({video_url})"
    )

    try:
        await sm.delete()
    except Exception:

```

```

        pass

        if thumb:
            sent = await
            send_photo_safe(c.bot, cid, thumb,
                           caption=caption,
                           parse_mode=ParseMode.MARKDOWN)
            if not sent:
                pass
            else:
                await
            u.message.reply_text(caption,
                                parse_mode=ParseMode.MARKDOWN,
                                reply_to_message_id=u.message.message_id)

        except Exception as e:
            logger.error(f"[yt_handler] {e}")
            try:
                await sm.edit_text(f"🐱 Error:
`{str(e)[:60]}`",
                                parse_mode=ParseMode.MARKDOWN)
            except Exception:
                pass

    async def block_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
        """Owner-only. /block <pack_name OR
t.me/addstickers/ URL> bans a sticker
pack."""
        if not u.message:
            return
        try:
            if not is_owner(u.effective_user.id

```

```

if u.effective_user else 0):
    await u.message.reply_text("🚫
Owner only.")
    return
    parts =
u.message.text.split(maxsplit=1)
    if len(parts) < 2:
        await u.message.reply_text("⚠️
Usage: `/block pack_name` or `/block
https://t.me/addstickers/packname`)
        return
        pack_input = parts[1].strip()
        pack_name =
pack_input.split("t.me/addstickers/")
[-1].strip("/") if "t.me/addstickers/" in
pack_input else pack_input
        await ban_sticker_pack(pack_name)
        await u.message.reply_text(
            f"🚫 *Pack blocked:*
            `{pack_name}`\n"
            f"Any sticker from this pack
            sent by anyone in this group will now be
            auto-deleted.",
            parse_mode=ParseMode.MARKDOWN
        )
    except Exception as e:
        logger.error(f"[block] {e}")
        await u.message.reply_text(f"❌
Error: `{str(e)[:60]}`")

async def monitor_private_chat(u: Update,
c: ContextTypes.DEFAULT_TYPE):
    """
    Plain AI chat for DMs. No

```

```

secretary/business logic anymore – every
    private text message just gets an AI
    reply, with a sticker from the
        MAIN pack attached to every single
    response (private-chat rule).
    """
    if not u.message or
u.effective_chat.type != "private":
        return
    try:
        text = (u.message.text or
u.message.caption or "").strip()
        if not text or
text.startswith("/"):
            return

        ss_match = re.match(r"^ss\s+(\S+)",
text, re.IGNORECASE)
        if ss_match:
            target = ss_match.group(1)
            if not
target.startswith("http"):
                target = "https://" +
target
            await _screenshot_website(u, c,
target)
            return

        search_match =
re.match(r"^(?:search|google)\s+(.+)",
text, re.IGNORECASE)
        if search_match:
            await _run_search_and_reply(u,
c, search_match.group(1).strip())

```

```

        return

        uid = u.effective_user.id
        user_name =
get_user_name(u.effective_user)
        memory = await get_user_memory(uid)
        mem_ctx =
build_memory_context(memory)
        hist_ctx =
build_chat_history_context(memory)
        system = f"{CHAT_PROMPT}\nThe
user's name is {user_name}.{mem_ctx}
{hist_ctx}"
        reply = await ai(system, text,
f"Hey {user_name}! 🐾", max_tok=140)

        try:
            await
u.message.reply_text(reply,
reply_to_message_id=u.message.message_id)
        except Exception:
            pass

        await append_chat_history(uid,
text, reply)

        stick = await
get_random_sticker_from(next_ai_sticker_pac
k())
        if stick:
            try:
                await
c.bot.send_sticker(chat_id=u.effective_chat
.id, sticker=stick)

```

```

        except Exception:
            pass
    except Exception as e:
        logger.error(f"
[monitor_private_chat] {e}")

async def monitor_ghost_mode(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message or
u.effective_chat.type == "private":
        return
    text = (u.message.text or "").strip()
    if not text:
        return

    bot_username =
bot_status.get("username", "")
    text_lower = text.lower()

    mentioned = False
    if bot_username and f"@{bot_username}"
in text_lower:
        mentioned = True
    if "@smartbeluga_bot" in text_lower:
        mentioned = True
    if not mentioned:
        return

    msg_content = re.sub(r"@\\w+", "", text,
flags=re.IGNORECASE).strip()
    if not msg_content:
        return

    try:

```

```

        uid = u.effective_user.id
        user_name =
get_user_name(u.effective_user)
        memory = await get_user_memory(uid)
        mem_ctx =
build_memory_context(memory)
        hist_ctx =
build_chat_history_context(memory)
        system = f"{CHAT_PROMPT}\nThe
user's name is {user_name}.{mem_ctx}
{hist_ctx}"
        reply = await ai(system,
msg_content, f"Hey {user_name}! 🐾",
max_tok=140)
        await u.message.reply_text(reply,
reply_to_message_id=u.message.message_id)
        await append_chat_history(uid,
msg_content, reply)
    except Exception as e:
        logger.error(f"[monitor_ghost_mode]
{e}")

ai_reply_counter = {}
_ai_sticker_toggle = {"n": 0}

def next_ai_sticker_pack() -> str:
    """Alternates between the main pack and
the new pybelu pack on every AI
reply that attaches a sticker – main
pack first, then pybelu, then main..."""
    _ai_sticker_toggle["n"] += 1
    return STICKER_PACK_MAIN if
_ai_sticker_toggle["n"] % 2 == 1 else
STICKER_PACK_ALT

```

```

async def monitor_group(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message or not
u.effective_user or
u.effective_user.is_bot:
        return
    if u.effective_chat.type == "private":
        return
    try:
        uid, cid, now =
u.effective_user.id,
str(u.effective_chat.id), datetime.now()

        if u.message.sticker:
            pack_of_sticker =
getattr(u.message.sticker, "set_name",
None)
            if
is_pack_banned(pack_of_sticker):
                try:
                    await
u.message.delete()
                except Exception:
                    pass
            return

        spam_tracker.setdefault(uid, [])
        spam_tracker[uid] = [t for t in
spam_tracker[uid] if now - t <
timedelta(seconds=2)]
        spam_tracker[uid].append(now)
        if len(spam_tracker[uid]) >= 4:
            try: await u.message.delete()

```



```

        except Exception: pass
        return

        db.setdefault("seen",
        {}).setdefault(cid, {})[str(uid)] = {
            "id": uid, "un":
u.effective_user.username, "n":
u.effective_user.first_name or "User"
        }
        counts = db.setdefault("counts",
        {})
        counts[cid] = counts.get(cid, 0) +
1

        if counts[cid] % 14 == 0:
            stick_safe = await
get_random_sticker_from(STICKER_PACK_SAFE)
            if stick_safe:
                try:
                    await
c.bot.send_sticker(chat_id=u.effective_chat
.id, sticker=stick_safe)
                except Exception:
                    pass

            text = (u.message.text or
u.message.caption or "").strip()
            if not text:
                bot_status["message_count"] +=
1

            return

            ss_match = re.match(r"^ss\s+(\S+)",
text, re.IGNORECASE)

```

```

        if ss_match:
            target = ss_match.group(1)
            if not
target.startswith("http"):
                target = "https://" +
target
                await _screenshot_website(u, c,
target)
                bot_status["message_count"] +=
1
                return

            search_match =
re.match(r"^(?:search|google)\s+(.+)",
text, re.IGNORECASE)
            if search_match:
                await _run_search_and_reply(u,
c, search_match.group(1).strip())
                bot_status["message_count"] +=
1
                return

            if text.startswith("/"):
                bot_status["message_count"] +=
1
                return

            bot_username =
bot_status.get("username", "")
            text_low = text.lower()
            contains_beluga = "beluga" in
text_low
            contains_username =
bool(bot_username) and (bot_username in

```

```

text_low or f"@{bot_username}" in text_low)
    is_reply =
(u.message.reply_to_message and
u.message.reply_to_message.from_user
    and
u.message.reply_to_message.from_user.id ==
c.bot.id)

    if contains_beluga or
contains_username or is_reply:
        try: await
asyncio.wait_for(c.bot.send_chat_action(u.e
ffective_chat.id, "typing"), timeout=4.0)
        except Exception: pass

        emoji = await ai_emoji(text)
        try: await safe_react(c.bot,
u.effective_chat.id, u.message.message_id,
emoji)
        except Exception: pass

        user_name =
get_user_name(u.effective_user)
        memory = await
get_user_memory(uid)
        mem_ctx =
build_memory_context(memory)
        hist_ctx =
build_chat_history_context(memory)
        system = f"{CHAT_PROMPT}\nThe
user's name is {user_name}.{mem_ctx}
{hist_ctx}"
        reply = await ai(system, text,
f"Hey {user_name}! 🐾", max_tok=140)

```

```

        try:
            await
            u.message.reply_text(reply,
            reply_to_message_id=u.message.message_id)
        except Exception:
            pass

        await append_chat_history(uid,
        text, reply)

        ai_reply_counter[cid] =
        ai_reply_counter.get(cid, 0) + 1
        if ai_reply_counter[cid] % 2 ==
        0:
            stick = await
            get_random_sticker_from(next_ai_sticker_pac
            k())

            if stick:
                try:
                    await
                    c.bot.send_sticker(chat_id=u.effective_chat
                    .id, sticker=stick)
                except Exception:
                    pass

            bot_status["message_count"] += 1
        except Exception as e:
            logger.error(f"[monitor_group]
            {e}")

    async def _health(req):
        up = int((datetime.now() -
        bot_status["start_time"]).total_seconds())

```

```

        return web.json_response({"status":
    "healthy", "uptime_seconds": up, "running":
    bot_status["running"], "messages":
    bot_status["message_count"], "version":
    "11.4.0"})

    async def _ping(req):
        return web.json_response({"pong": True,
    "ts": datetime.now().isoformat()})

    async def start_http(port):
        aio = web.Application()
        aio.router.add_get("/", _ping)
        aio.router.add_get("/health", _health)
        aio.router.add_get("/ping", _ping)
        runner = web.AppRunner(aio)
        await runner.setup()
        await web.TCPSite(runner, "0.0.0.0",
    port).start()
        logger.info(f"HTTP server up on
    0.0.0.0:{port}")
        return runner

    async def error_handler(update: object,
    context: ContextTypes.DEFAULT_TYPE):
        err = context.error
        if isinstance(err, (NetworkError,
    Timeout, Forbidden)):
            return
        if isinstance(err, Conflict):
            # Expected for a few seconds during
    a Render rolling deploy, when the
            # old and new instance both briefly
    poll with the same bot token.

```

```

        # PTB retries internally – just log
        it quietly, don't alert/crash.
        logger.warning("[Conflict] Another
        getUpdates instance is polling – will retry
        automatically.")
        return
        if isinstance(err, RetryAfter):
            await asyncio.sleep(err.retry_after
+ 1); return
        if isinstance(err, BadRequest) and "not
        modified" in str(err).lower():
            return
            bot_status["error_count"] += 1
            logger.error(f"[Err] {err}")
            if OWNER_ID:
                try:
                    await
context.bot.send_message(chat_id=OWNER_ID,
text=f"⚠️ *Error:* `{str(err)[:150]}`",
parse_mode=ParseMode.MARKDOWN)
                except Exception:
                    pass

START_TEXT = (
    "Hi, human friend! 🐱 I\u2019m
    Beluga!\n\n"
    "Ready to chat?!! 🐱\n\n"
    "Just ask me anything – Science, Maths,
    Random Questions, or Something You\u2019re
    curious about...\n\n"
    "You Can Start With :\n"
    "\u201cHi Beluga, what is the Big Bang
    Theory?\u201d 🌟\n\n"
    "Your Turn, Poookie 🎀"

```

)

```
WHAT_I_CAN_DO_TEXT = (  
    "* ✨ Beluga's Command Book ✨*\n\n"  
    "* 🎮 Games & Fun*\n"  
    "`/quiz` - brain trivia\n"  
    "`/tictac` - tic tac toe\n"  
    "`/mine` - minesweeper\n"  
    "`/gay` `/couple` - daily fun  
picks\n\n"  
    "* 💰 Crypto Live*\n"  
    "`/price` - live coin price\n"  
    "`/topgainers` `/toplosers` - market  
movers\n"  
    "`/chart` - candlestick chart\n\n"  
    "* 📰 News*\n"  
    "`/news` - crypto headlines\n"  
    "`/ainews` - AI updates\n"  
    "`/technews` - tech world\n\n"  
    "* 🔍 Search & AI*\n"  
    "`/search` - web + Wikipedia search\n"  
    "`/bananalogic` - AI answer with  
image\n"  
    "`/yt` - YouTube video info\n"  
    "_@ mention me anytime to chat!_\n\n"  
    "* 🖼️ Image Tools*\n"  
    "`/qr` - QR code generator\n"  
    "`/scanqr` - scan a QR code\n"  
    "`/resize` `/compress` - image tools\n"  
    "`/watermark` - add a watermark\n"  
    "`/imginfo` - image details\n\n"  
    "* 🏆 Leaderboard*\n"  
    "`/lb` - view rankings\n"  
    "`/gm` `/nw` `/pump` `/dump` - admin
```

```

tools\n\n"
    "**💀 Extras*\n"
    "`/model` – pick AI engine (admin)\n"
    "`/block` – ban a sticker pack
(admin)\n"
    "`/clearmemory` – wipe memory
(admin)\n"
    "`/workflow` – how I think"
)

def _start_main_menu_kb() ->
InlineKeyboardMarkup:
    bot_username =
bot_status.get("username", "")
    kidnap_url =
f"https://t.me/{bot_username}?
startgroup=true&admin=post_messages+edit_me
ssages+delete_messages+restrict_members+inv
ite_users+pin_messages" if bot_username
else KIDNAP_ME_URL
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("💋 KIDNAP ME
💋", url=kidnap_url)],
        [InlineKeyboardButton("💖 UPDATES
CHANNEL 💖", url=UPDATES_CHANNEL)],
        [InlineKeyboardButton("🐱 WHAT I
CAN DO ?", callback_data="menu:whatido")],
        [InlineKeyboardButton("👁️
WORKFLOW", callback_data="menu:workflow")],
    ])

def _back_only_kb() ->

```



```

InlineKeyboardMarkup:
    return
InlineKeyboardMarkup([[InlineKeyboardButton
("BACK 😞", callback_data="menu:back")]])

async def start_handler(u: Update, c:
ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    kb = _start_main_menu_kb()
    sent = await send_photo_safe(
        c.bot, u.effective_chat.id,
START_MENU_IMAGE_PAGE,
        caption=START_TEXT,
parse_mode=ParseMode.MARKDOWN,
reply_markup=kb
    )
    if not sent:
        # send_photo_safe already sent the
text fallback if photo failed;
        # this only fires if BOTH photo and
text somehow failed, so retry once more.
        try:
            await
u.message.reply_text(START_TEXT,
parse_mode=ParseMode.MARKDOWN,
reply_markup=kb)
        except Exception as e:
            logger.error(f"[start_handler]
Total failure: {e}")

async def workflow_handler(u: Update, c:

```

```

ContextTypes.DEFAULT_TYPE):
    if not u.message:
        return
    sent = await send_photo_safe(
        c.bot, u.effective_chat.id,
        WORKFLOW_IMAGE_PAGE,
        caption=WORKFLOW_TEXT,
        parse_mode=ParseMode.MARKDOWN
    )
    if not sent:
        try:
            await
u.message.reply_text(WORKFLOW_TEXT,
        parse_mode=ParseMode.MARKDOWN)
        except Exception as e:
            logger.error(f"
[workflow_handler] Final fallback failed:
{e}")

```

```

async def start_menu_callback(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """
    Handles all 3 submenu buttons + the
    BACK button on /start's inline menu.
    Edits the existing message in place
    (text or caption, whichever the
    message actually has) instead of
    sending new messages every tap.
    """
    q = update.callback_query
    if not q:
        return

```

```

try:
    await q.answer()
    action = q.data.split(":", 1)[1]

    if action == "whatido":
        target_text, target_kb =
WHAT_I_CAN_DO_TEXT, _back_only_kb()
    elif action == "workflow":
        target_text, target_kb =
WORKFLOW_TEXT, _back_only_kb()
    elif action == "back":
        target_text, target_kb =
START_TEXT, _start_main_menu_kb()
    else:
        return

    try:
        if q.message.photo:
            await
q.edit_message_caption(caption=target_text,
parse_mode=ParseMode.MARKDOWN,
reply_markup=target_kb)
        else:
            await
q.edit_message_text(text=target_text,
parse_mode=ParseMode.MARKDOWN,
reply_markup=target_kb)
    except Exception as e:
        logger.warning(f"
[start_menu_callback] Edit failed ({e}),
sending fresh message instead.")
        await
context.bot.send_message(chat_id=q.message.
chat_id, text=target_text,

```

```

parse_mode=ParseMode.MARKDOWN,
reply_markup=target_kb)
    except Exception as e:
        logger.error(f"
[start_menu_callback] {e}")

MEMORY_TTL_MESSAGES = 6

def _mongo_available() -> bool:
    return mongo_memory_col is not None

async def get_user_memory(user_id) -> dict:
    """Fetch a user's rolling chat memory
    from MongoDB. Returns {"messages": []} if
    none/unavailable."""
    if not _mongo_available():
        return {"messages": []}
    try:
        doc = await
mongo_memory_col.find_one({"_id":
str(user_id)})
        return doc if doc else {"messages":
[]}
    except Exception as e:
        logger.error(f"[mongo
get_user_memory] {e}")
        return {"messages": []}

async def save_user_memory(user_id,
memory_data: dict) -> bool:
    """Overwrite a user's full memory doc

```

```

in MongoDB."""
    if not _mongo_available():
        return False
    try:
        await
    mongo_memory_col.replace_one({"_id":
str(user_id)}, memory_data, upsert=True)
        return True
    except Exception as e:
        logger.error(f"[mongo
save_user_memory] {e}")
        return False

async def append_chat_history(user_id,
user_text: str, bot_reply: str) -> bool:
    """
        Append one (user, bot) exchange to a
        user's rolling 6-message memory.
        Once the count exceeds 6 messages, the
        entire history for that user is
        auto-cleared (per spec) so the next
        message starts fresh.
    """
    if not _mongo_available():
        return False
    try:
        doc = await
    mongo_memory_col.find_one({"_id":
str(user_id)}) or {"messages": []}
        messages = doc.get("messages", [])
        messages.append({"role": "user",
"text": user_text[:500], "ts":
datetime.utcnow().isoformat()})
        messages.append({"role": "bot",

```

```

"text": bot_reply[:500], "ts":
datetime.utcnow().isoformat())
    if len(messages) > 6:
        messages = []
    await
mongo_memory_col.replace_one({"_id":
str(user_id)}, {"_id": str(user_id),
"messages": messages}, upsert=True)
    return True
except Exception as e:
    logger.error(f"[mongo
append_chat_history] {e}")
    return False

def build_chat_history_context(memory:
dict) -> str:
    """Turn stored rolling messages into a
short context block for the AI prompt."""
    messages = memory.get("messages", [])
    if not messages:
        return ""
    lines = []
    for m in messages[-6:]:
        who = "User" if m.get("role") ==
"user" else "You (Beluga)"
        lines.append(f"{who}:
{m.get('text', '')}")
    return "\n\nRecent conversation:\n" +
"\n".join(lines)

def build_memory_context(memory: dict) ->
str:
    """Kept for compatibility with callers;
rolling history covers this now."""

```

```

        return ""

    async def delete_user_memory(user_id) ->
    bool:
        if not _mongo_available():
            return False
        try:
            await
            mongo_memory_col.delete_one({"_id":
            str(user_id)})
            return True
        except Exception as e:
            logger.error(f"[mongo
            delete_user_memory] {e}")
            return False

    async def clear_all_memory() -> tuple:
        """Wipe every user's memory doc from
        MongoDB. Returns (deleted_count,
        failed)."""
        if not _mongo_available():
            return (0, 1)
        try:
            result = await
            mongo_memory_col.delete_many({})
            return (result.deleted_count, 0)
        except Exception as e:
            logger.error(f"[mongo
            clear_all_memory] {e}")
            return (0, 1)

    async def clearmemory_handler(u: Update, c:
    ContextTypes.DEFAULT_TYPE):
        if not u.message:

```

```

        return
    if not is_owner(u.effective_user.id if
u.effective_user else 0):
        await u.message.reply_text("🚫
Owner only.")
        return
    status_msg = await
u.message.reply_text("CLEARING MEMORY
🔧 .....")
    deleted, failed = await
clear_all_memory()
    if failed == 0:
        result_text = f"✅ Memory cleared!
Removed {deleted} user record(s) from
MongoDB."
    else:
        result_text = "⚠️ MongoDB not
available or clear failed – check logs."
    try:
        await
status_msg.edit_text(f"CLEARING MEMORY
🔧 .....\\n\\n{result_text}")
    except Exception:
        await
u.message.reply_text(result_text)

async def main():
    logger.info("STARTING BELUGA BOT
v11.4.0")
    http_runner = await
start_http(HTTP_PORT)
    await asyncio.sleep(0.3)

    app =

```



```
TGApp.builder().token(BOT_TOKEN).build()

    await load_persistent_data()

    app.add_handler(CommandHandler("start",
start_handler))

app.add_handler(CommandHandler("workflow",
workflow_handler))
    app.add_handler(CommandHandler("price",
crypto_price_handler))

app.add_handler(CommandHandler(["topgainers",
"toplosers"], crypto_movers_handler))

app.add_handler(CommandHandler(["chart",
"chart5m", "chart15m", "chart1h",
"chart4h", "chart1d"],
crypto_chart_handler))
    app.add_handler(CommandHandler("news",
lambda u, c: execute_news_flow(u, c,
"crypto", "Crypto News"))))

app.add_handler(CommandHandler("ainews",
lambda u, c: execute_news_flow(u, c, "ai",
"AI News"))))

app.add_handler(CommandHandler("technews",
lambda u, c: execute_news_flow(u, c,
"tech", "Tech News"))))

app.add_handler(CommandHandler("search",
search_handler))
    app.add_handler(CommandHandler("ss",
```

```
ss_handler))

app.add_handler(CommandHandler("bananalogic",
    bananalogic_handler))
    app.add_handler(CommandHandler("yt",
yt_handler))
    app.add_handler(CommandHandler("qr",
qr_generate_handler))

app.add_handler(CommandHandler("scanqr",
qr_scan_handler))

app.add_handler(CommandHandler("resize",
    lambda u, c: img_handler(u, c, "resize")))

app.add_handler(CommandHandler("compress",
    lambda u, c: img_handler(u, c,
"compress")))

app.add_handler(CommandHandler("watermark",
watermark_handler))

app.add_handler(CommandHandler("imginfo",
    lambda u, c: img_handler(u, c, "info")))
    app.add_handler(CommandHandler("quiz",
quiz_handler))
    app.add_handler(CommandHandler(["lb",
"leaderboard"], lb_handler))
    app.add_handler(CommandHandler("nw",
nw_handler))
    app.add_handler(CommandHandler(["pump",
"dump"], pump_dump_handler))

app.add_handler(CommandHandler("tictac",
```

```
tictac_handler))
    app.add_handler(CommandHandler("mine",
mine_handler))
    app.add_handler(CommandHandler("gm",
gm_handler))
    app.add_handler(CommandHandler(["gay",
"couple"], fun_dispatcher))
    app.add_handler(CommandHandler("block",
block_handler))

app.add_handler(CommandHandler("clearmemory
", clearmemory_handler))
    app.add_handler(CommandHandler("model",
model_command_handler))

app.add_handler(CallbackQueryHandler(ttt_ca
llback, pattern=r"^ttt:"))

app.add_handler(CallbackQueryHandler(gm_cal
lback, pattern=r"^gm:"))

app.add_handler(CallbackQueryHandler(mine_c
allback, pattern=r"^mine:"))

app.add_handler(CallbackQueryHandler(waterm
ark_callback, pattern=r"^wm:"))

app.add_handler(CallbackQueryHandler(model_
callback, pattern=r"^model:"))

app.add_handler(CallbackQueryHandler(start_
menu_callback, pattern=r"^menu:"))
```

```
app.add_handler(PollAnswerHandler(poll_answer_handler))
```

```
app.add_handler(MessageHandler(filters.Sticker.ALL & filters.ChatType.GROUPS,  
monitor_group), group=0)
```

```
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND &  
filters.ChatType.PRIVATE,  
monitor_private_chat), group=1)
```

```
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND &  
filters.ChatType.GROUPS,  
monitor_ghost_mode), group=2)
```

```
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND &  
filters.ChatType.GROUPS, monitor_group),  
group=3)
```

```
app.add_error_handler(error_handler)
```

```
await app.initialize()  
await app.start()
```

```
await load_sticker_pack(app.bot,  
STICKER_PACK_MAIN)  
await load_sticker_pack(app.bot,  
STICKER_PACK_SAFE)  
await load_sticker_pack(app.bot,  
STICKER_PACK_ALT)
```

```

    await save_all_data()

    try:
        me = await app.bot.get_me()
        bot_status["username"] =
me.username.lower()
        logger.info(f"Bot identity:
@{me.username}")
    except Exception as e:
        logger.warning(f"[Startup get_me]
{e}")

    # Safety: explicitly drop any stale
webhook + pending updates before
    # polling starts, so an old instance's
leftover session can't cause a
    # getUpdates Conflict with this one.
    try:
        await
app.bot.delete_webhook(drop_pending_updates
=True)
    except Exception as e:
        logger.warning(f"[Startup
delete_webhook] {e}")

    await
app.updater.start_polling(drop_pending_upda
tes=True, allowed_updates=[])
    bot_status["running"] = True
    logger.info("Beluga Bot is running")

    stop_evt = asyncio.Event()
    try:
        import signal

```

```

        loop = asyncio.get_running_loop()

loop.add_signal_handler(signal.SIGTERM,
stop_evt.set)

loop.add_signal_handler(signal.SIGINT,
stop_evt.set)
    except Exception:
        pass

    cleanup_task =
asyncio.create_task(cleanup_expired_games()
)
    sync_task =
asyncio.create_task(periodic_sync())
    exchange_task =
asyncio.create_task(init_exchange_async())

    await stop_evt.wait()
    logger.info("Shutting down...")
    cleanup_task.cancel()
    exchange_task.cancel()
    sync_task.cancel()
    bot_status["running"] = False
    for fn in [app.updater.stop, app.stop,
app.shutdown, http_runner.cleanup]:
        try:
            await fn()
        except Exception:
            pass

if __name__ == "__main__":
    try:
        asyncio.run(main())

```

```
except KeyboardInterrupt:
    logger.info("Interrupted by user")
except Conflict:
    # Another instance (usually the
previous Render deploy, briefly
    # overlapping during a rolling
deploy) was polling with the same
    # token. This is transient – exit
cleanly so Render restarts this
    # instance rather than logging it
as a hard crash.
    logger.warning("[Startup] Conflict:
another instance was polling. Exiting to
let Render restart cleanly.")
    sys.exit(0)
except Exception as e:
    logger.critical(f"Fatal: {e}")
    sys.exit(1)
```